

Técnico Superior en Desarrollo de Aplicaciones Web
TatuSys: gestión de estudio de tatuaje

Proyecto presentado por:	Santiago Múnera Preciado Patricia Victoria Sanz López Laura Vinseiro Gutiérrez
Tipo de trabajo:	Proyecto de emprendimiento (Desarrollo de software)
Tutor/a:	Tomás Escudero Delgado
Fecha:	06/04/2026

Resumen

TatuSys es una aplicación de administración creada para un estudio de tatuaje con dos objetivos primordiales: automatizar y simplificar la realización de presupuestos y gestionar las citas dentro de un sistema unificado (un dashboard de administración). Su desarrollo pretende llevar a la práctica los conocimientos adquiridos durante el ciclo y cubrir las carencias observadas en el sector, donde es habitual emplear múltiples herramientas de diversos proveedores para las distintas tareas del negocio (es decir, presupuestos, comunicación con clientes o agendas de trabajo).

El diseño consta de un backend adecuado al tipo de datos que se necesitan gestionar, junto a un frontend con dos vertientes: un sitio web sencillo en el que los clientes finales pueden solicitar cita y presupuesto para tatuarse, retocar un diseño o cubrirlo; acompañado de un panel de control que permite al dueño del estudio recibir y gestionar solicitudes, aprobar los presupuestos autogenerados, revisar las próximas citas y editar los horarios de trabajo, entre otros ajustes.

Palabras clave: presupuestos, citas, dashboard, tatuaje.

Abstract

TatuSys is a management application designed for a tattoo studio with two primary objectives: to automate and simplify the quoting process and to centralize appointment management within a unified dashboard.

This project seeks to apply the technical skills acquired during the academic program while addressing a common deficiency in the sector: the reliance on a variety of tools and providers for day-to-day business tasks such as quoting, client communication, and work scheduling.

The system architecture consists of a backend tailored to the specific data requirements of the studio and a dual frontend. This consists of a public-facing website that allows clients to request appointments and quotes for new tattoos, touch-ups, or cover-ups.; along with an administrative control panel that enables studio owners and staff to process these requests, validate auto-generated quotes, track upcoming appointments, and manage work schedules.

Keywords: quoting, appointment scheduling, dashboard, tattoo studio management.

Índice de contenidos

1. INTRODUCCIÓN	8
1.1. Justificación	8
2. ESTADO DEL TEMA TRATADO EN EL MOMENTO ACTUAL	9
3. OBJETIVOS Y RETOS A CONSEGUIR	10
4. METODOLOGÍA A EMPLEAR	11
5. Desarrollo del proyecto.....	15
5.1. Introducción: estudio y análisis de la aplicación	15
5.2. Aspecto visual del proyecto.....	16
5.2.1. Imagen de marca y guía de estilo:.....	16
5.2.2. Prototipado del proyecto y flujo de uso.....	18
5.3. Desarrollo del frontend (sitio web del estudio)	21
5.3.1. Arquitectura y organización de los componentes del sitio web	21
5.4. Arquitectura del backend	27
5.4.1. Diseño y modelado de la base de datos.....	27
5.4.1.1 Esquema de la base de batos	27
5.4.1.2 Entidades JPA y mapeo Objeto-Relacional	29
5.4.2 Implementación de la API REST con Spring Boot.....	31
5.4.2.1 Arquitectura de capas del sistema	31
5.4.2.2 Capa de controladores REST	33
5.4.2.3 Capa de servicios y lógica de negocio	35
5.4.2.4 Capa de repositorios y acceso a datos	36
5.4.2.5 Sistema de seguridad y autenticación	36
5.4.2.6 DTOs y transformación de datos.....	39
5.4.3. Generación y envío de facturas en formato PDF	40
5.5. Desarrollo del frontend (dashboard del tatuador).....	45

5.5.1.	Introducción al interfaz del tatuador	45
5.5.2.	Página de Solicitudes	48
5.5.3.	Home (página de citas confirmadas)	53
5.5.4.	Calendario.....	57
5.5.5.	Trabajadores (configuración del personal)	57
5.5.6.	Configuración.....	60
6.	Conclusiones.....	62
6.1.	Conclusiones generales del proyecto.....	62
6.1.1.	Conclusiones comunes	62
6.1.2.	Conclusiones a título personal (Patricia)	62
6.1.3.	Conclusiones a título personal (Santiago)	63
6.1.4.	Conclusiones a título personal (Laura)	64
6.2.	Limitaciones y prospectiva	65
7.	REFERENCIAS BIBLIOGRÁFICAS	66
8.	ENLACE AL REPOSITORIO DEL PROYECTO	67
9.	ANEXOS	68
Anexo A.	Guía de estilo (Doc. adjunto).....	68
Anexo B.	Wireflow del sitio web (Doc. adjunto).....	68
Anexo C.	Wireflow del dashboard (Doc. adjunto).....	68
Anexo D.	Mockup sitio web (desktop) (Doc. adjunto).....	68
Anexo E.	Mockup sitio web (móvil) (Doc. adjunto).....	68
Anexo F.	Mockup dashboard - admin (escritorio) (Doc. adjunto).....	68
Anexo G.	Mockup dashboard - admin (móvil) (Doc. adjunto).....	68
Anexo H.	Mockup dashboard – trab. (escritorio) (Doc. adjunto).	68
Anexo I.	Mockup dashboard – trab. (móvil) (Doc. adjunto).....	68

Índice de figuras

Figura 1: Arquitectura de software del proyecto (fase de pruebas).	12
Figura 2: Arquitectura de software del proyecto (posterior a fase de pruebas).	12
Figura 3. Verificación de visibilidad la paleta de color en Coolors.com	17
Figura 4. Captura de sección de tipografía y color en la guía de estilo.....	17
Figura 5. Logotipo diseñado para el estudio INK&CO.	18
Figura 6. Fragmento de bocetos en papel para el diseño del sitio web.	19
Figura 7. Fragmento del wireflow elaborado para el sitio web.	19
Figura 8. Captura del mockup de Figma (versión de escritorio) del sitio.....	20
Figura 9. Estructura de carpetas del proyecto.	22
Figura 10. Ejemplos de validaciones y pop-up de ayuda del formulario.	23
Figura 11. Selector de fecha y hora del formulario de reserva de citas.....	24
Figura 12. Recuadro para modificación o cancelación de cita.	25
Figura 13. Ejemplo de uso de Bootstrap en la página de inicio del sitio web.....	26
Figura 14. Ejemplo de uso de media queries en el sitio web.....	26
Figura 15. Diagrama entidad-relación generado por ingeniería inversa desde la base de datos MySQL.....	27
Figura 16. Ejemplo de mapeo en la entidad Cita.	29
Figura 17. Ejemplo de mapeo @GeneratedValue en la entidad Cita.	30
Figura 18. Ejemplo de mapeo @Enumerated en la entidad Cita.....	30
Figura 19. Ejemplo de mapeo @Temporal en la entidad Cita.	31
Figura 20. Estructura de paquetes.	31
Figura 21. Ejemplo de enums de los estados.	36
Figura 22. Ejemplo de repositorio con herencia de JpaRepository.....	36
Figura 23. Imagen del flujo asíncrono del software.	43

Figura 24. Imagen del buzón de envío del estudio.....	43
Figura 25. Aspecto del pdf de la factura.....	44
Figura 26. Captura del formulario de inicio de sesión del dashboard.	46
Figura 27. Recuadro de recuperación de contraseña.	46
Figura 28. Recuadro de nueva contraseña.	47
Figura 29. Visualización del menú de navegación en escritorio (izq.) y móvil (der.).	47
Figura 30. Captura del apartado “Solicitudes Pendientes” del dashboard.....	48
Figura 31. Vista del recuadro de edición/visión de los detalles de una solicitud.....	50
Figura 32. Vista de las solicitudes revisadas en el dashboard.....	50
Figura 33. Vista del recuadro de información detallada de una solicitud revisada.	51
Figura 34. Imagen del cuerpo de un correo electrónico en la fase de pruebas.....	52
Figura 35. Captura de los recuadros de información y datos de pago disponibles al usar el enlace de pago.....	53
Figura 36. Recuadros de notificación de novedades en el Home del dashboard.	53
Figura 37. Vista de la tabla de próximas citas y las notificaciones en la zona inferior.	54
Figura 38. Vista del apartado para finalizar una cita.....	55
Figura 39. Página de creación de factura por petición del cliente.....	56
Figura 40. Página de inserción de datos para la factura.	56
Figura 41. Vista del calendario del dashboard.	57
Figura 42. Vista de la sección de gestión de los trabajadores.....	58
Figura 43. Vista del recuadro de alta de nuevos trabajadores.....	58
Figura 44. Vista del recuadro de modificación de datos de un trabajador.....	59
Figura 45. Vista del recuadro de eliminación de un trabajador.....	59
Figura 46. Vista del recuadro de modificación de datos del administrador.	60
Figura 47. Vista del recuadro de edición de los precios del establecimiento.....	61

1. INTRODUCCIÓN

1.1. Justificación

La idea inicial del proyecto surge de la observación de un problema real en un negocio conocido. Se trata de un estudio en el que trabajan varios profesionales, y a través del contacto personal y diversas conversaciones, uno de ellos comunica los problemas de gestión que observa en su jornada. Sus principales quejas son, por un lado, la dificultad de dar presupuestos precisos (muchas veces se olvidan pequeños costes de material, o no se preguntan todos los detalles al cliente y el precio final que se fija supone pequeñas pérdidas) y, por otro, la complejidad de gestionar las citas. Cuando alguien pide un presupuesto, suelen comunicarse a través de WhatsApp, pero también reciben solicitudes por Instagram y por teléfono, y después tienen que usar Google Calendar para anotar las reservas. En muchas ocasiones hay olvidos, imprecisiones, citas solapadas porque el calendario no se actualiza al instante o personas que no se presentan porque no hay un sistema de recordatorios ni fianzas.

Esta situación llevó a plantear si con una aplicación sería posible solventar los problemas mediante la unificación de todas las tareas de gestión en un mismo lugar. La idea se vio reforzada más tarde al comprobar que no existe un servicio simplificado para este tipo de negocios: las opciones de las que disponen es dar citas y presupuestos por WhatsApp o Instagram, o incluir sus estudios o servicios en aplicaciones como Booksy, que funcionan como amalgamadores de estudios de belleza y son similares a un Airbnb, de modo que el negocio acaba listado junto a su competencia y pierde parte de sus ingresos por el porcentaje que se llevan estas aplicaciones.

Así pues, se concluye que no hay muchos ejemplos de aplicaciones que ofrezcan una funcionalidad parecida en el mercado en español, si bien existen algunos sistemas similares en el mercado angloparlante, por lo que parece una idea viable y con proyección de uso real.

2. ESTADO DEL TEMA TRATADO EN EL MOMENTO ACTUAL

Para poder estudiar el estado actual de productos similares al que presentamos, nos hemos centrado en sitios web relacionados con estudios de tatuaje y en sitios web orientados a la gestión de citas en diferentes ámbitos. Además, hemos buscado tanto en el mercado español como angloparlante. Hemos destacado algunos sitios web como ejemplos principales, que son los siguientes:

Enlaces de ofertas similares en mercado angloparlante:

- <https://www.venue.ink/tattoo-booking-forms>
- <https://inkdesk.app/>
- <https://calendly.com/scheduling>

Enlaces de aplicaciones y software similar en español:

- <https://studioapp.co/#product>
- <https://inkoru.com/caracteristicas.php>
- <https://tattoostudiosystem.com/#features>
- <https://booksy.com/es-es/>

En cuanto a las utilidades y funcionalidades, se centran básicamente en gestionar agendas, generar recordatorios automatizados y ofrecer sistemas de seguimiento. Son herramientas para centralizar citas y gestionar el problema de los clientes no presentados mediante fianzas por adelantado. Ninguno ofrece la creación de presupuestos como tal, pasan directamente a la creación de facturas finales. Por tanto, la generación de presupuestos sería un elemento diferenciador de nuestra aplicación en el mercado actual.

En cuanto al Stack utilizado, hemos usado la herramienta Wappalyzer para analizar los sitios web anteriormente mencionados. Todos, exceptuando uno, emplean el lenguaje PHP; todos los que tienen base de datos, usan MySQL. Aquellos que contienen un CMS, contienen *WordPress*. La mayoría utilizan frameworks JavaScript, aunque aquí sí que encontramos una gran variedad de opciones: React es el más utilizado, pero también vemos el uso de Redux, Angular o Vue.

A partir de aquí, las tecnologías son muy variadas en cuanto al resto de utilidades, como seguridad, cookies, análisis, servidores, etc.

3. OBJETIVOS Y RETOS A CONSEGUIR

Objetivos principales

- El primer objetivo que persigue la aplicación es facilitar la **creación de presupuestos** más precisos para los servicios que ofrecen los profesionales de tatuaje, de modo que estos no les generen pérdidas.
- El segundo objetivo, derivado del primero, es **automatizar la gestión de citas** a partir de la funcionalidad de creación y confirmación de presupuestos.

Objetivos secundarios

- Permitir a los trabajadores gestionar su agenda de trabajo, con vistas de las citas por día, semana o mes y un editor de horarios de trabajo.
- Permitir al dueño del estudio gestionar su plantilla de trabajadores con un sistema de altas, edición y bajas.
- Permitir la edición de precios de distintos servicios para que sirvan de base para los presupuestos automatizados.
- Simplificar la facturación mediante una funcionalidad de generación de facturas a partir de los presupuestos confirmados.
- Asegurar la realización de los servicios solicitados mediante una pasarela de pago que permita abonar fianzas y aumentar el compromiso de los clientes.

4. METODOLOGÍA A EMPLEAR

Para dotar de una estructura lógica a este apartado, hemos decidido usar las fases del ciclo de vida del software (planificación, análisis, diseño, implementación, pruebas) como hilo conductor de las explicaciones sobre las decisiones tomadas y las herramientas empleadas.

En lo que respecta a las fases de planificación y análisis, que en nuestro caso representan la etapa más temprana de conceptualización de nuestro proyecto, el punto de partida fue el planteamiento de cómo podrían utilizar los usuarios nuestra aplicación. Así pues, establecimos un mapa de pasos lógicos para el cliente final, desde su primer contacto con el sitio web del estudio de tatuajes hasta que concluye el servicio y sale por la puerta del establecimiento; y para el propietario y los empleados del negocio, desde que reciben la solicitud de un cliente hasta que completan el servicio y emiten la factura.

El sistema se ha desarrollado en torno a un proceso de reserva de citas que ayuda al usuario a comunicar lo que desea y que mejora su compromiso de asistencia mediante el pago de una fianza. Por parte del establecimiento, la solicitud simplifica la gestión de la agenda, agiliza la conversión de clientes mediante el envío de presupuestos rápidos y permite organizar las jornadas laborales con mayor control y menos olvidos u omisiones al proporcionar una agenda clara.

El flujo completo se desarrolla de la siguiente forma:

Solicitud desde el sitio web (formulario enviado) => Gestión desde el dashboard (revisión de la solicitud y el presupuesto, envío de un correo con enlace de fianza) => Cierre => el cliente paga la fianza y la cita se agenda o, si el cliente no paga la fianza, se elimina la solicitud y el horario se libera para otros posibles clientes.

A partir de esta conceptualización, establecimos una idea de la arquitectura de software necesaria para el proyecto, que se trataría de una arquitectura Cliente-Servidor con comunicación mediante API REST. A continuación, se muestra un esquema de esta, así como una explicación de sus elementos.

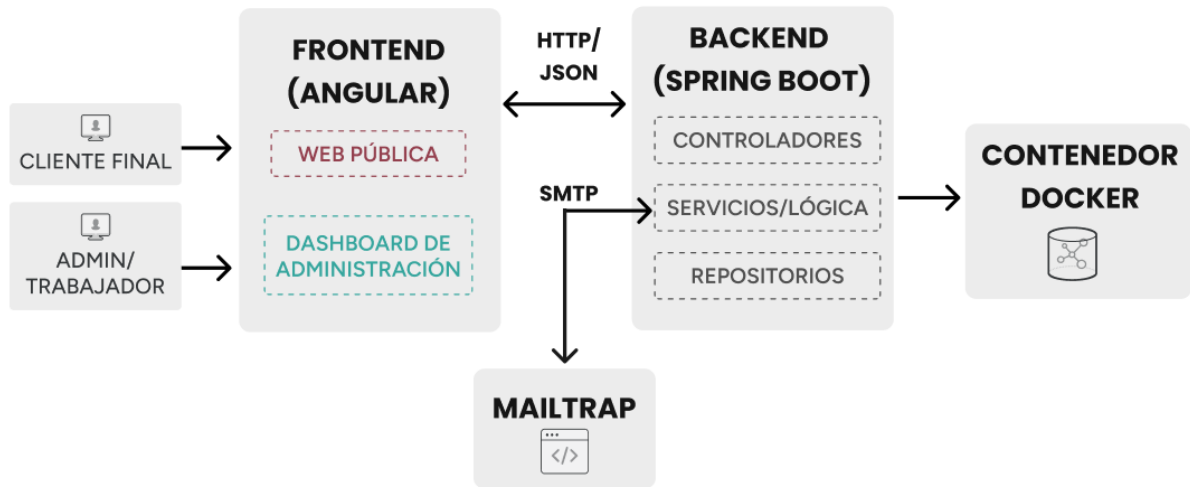


Figura 1: Arquitectura de software del proyecto (fase de pruebas).

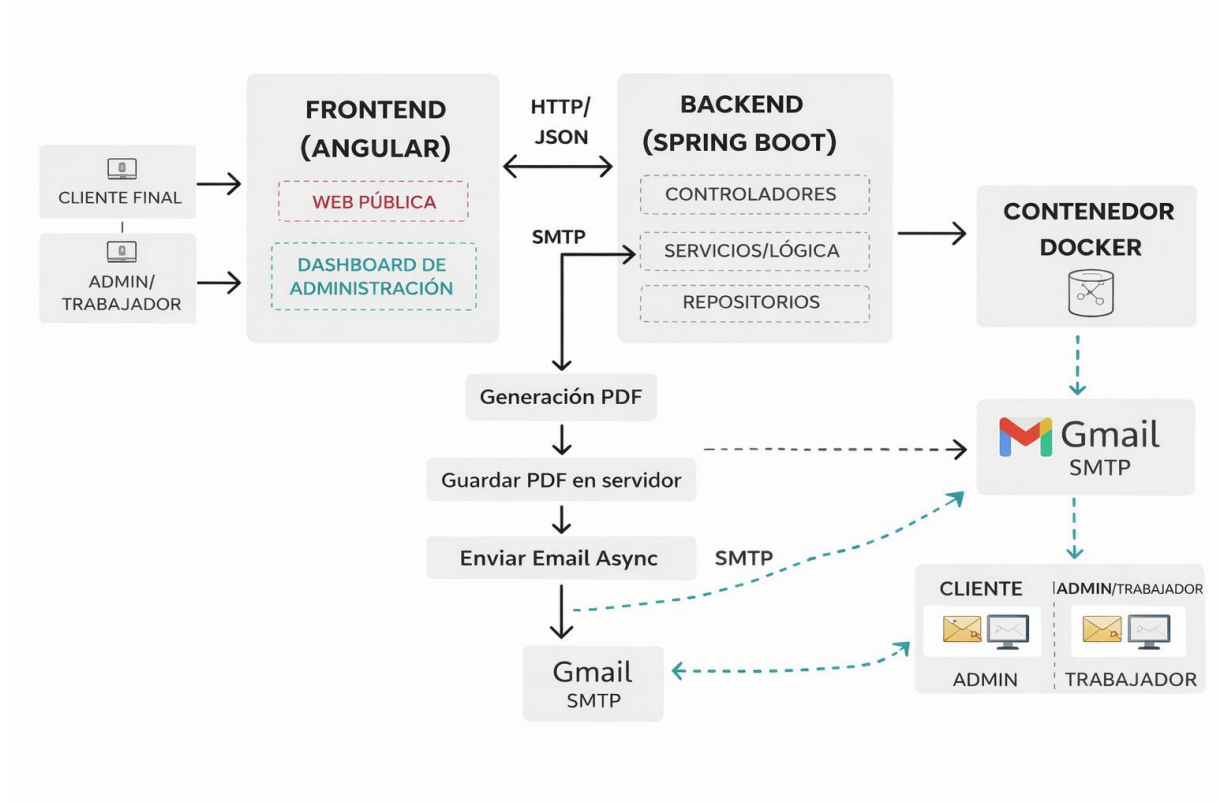


Figura 2: Arquitectura de software del proyecto (posterior a fase de pruebas).

Base de datos: los datos se almacenan en una BBDD relacional tipo MySQL, desplegada mediante un contenedor Docker que facilita la portabilidad del entorno de trabajo.

Backend: se desarrolla en Java con el framework Spring Boot. Sigue una arquitectura en capas con estructuración del código en controladores (RestController), servicios para la lógica de negocio (Service) y repositorios (Repository).

Frontend: se desarrolla en Angular con Typescript, HTML y CSS. Está formado por dos entornos (sitio web público y dashboard de administración), que se encuentran en la misma aplicación y cuya navegación se gestiona a través del enrutamiento.

La seguridad de esta arquitectura se implementa con una lógica de control de acceso que establece dos tipos de roles (administrador y trabajador) y exige un inicio de sesión con credenciales encriptadas para el dashboard.

Durante la fase de diseño, se ha partido de un estudio de tatuaje ficticio que sería nuestro cliente y las decisiones se han tomado en función de sus 'requisitos' y su 'imagen de marca', que hemos establecido en base a sitios web de referencia cuyo aspecto y comunicación de marca nos gustaba. La metodología de esta fase ha consistido en la elaboración de bocetos a mano alzada en papel para establecer estructuras básicas y disposiciones de elementos que nos parecían atractivas visualmente. Posteriormente, hemos utilizado Figma para trasladar los bocetos a algo más concreto y con un aspecto más profesional. Los pasos que hemos seguido, así como las herramientas y avances, se desarrollan con más detalle en el Subapartado 5 del presente documento.

Para la organización del desarrollo en sí se ha establecido un modelo de desarrollo tipo Scrum, caracterizado por *sprints* que dan prioridad a las funcionalidades más urgentes establecidas para cada fase. Esto nos ha permitido ir ajustando la implementación mediante la sincronización y comunicación diaria entre los miembros del equipo, pudiendo así adaptarnos a la disponibilidad de cada individuo para cumplir con los hitos de los bloques temporales de desarrollo. Así pues, se decidió ajustar los *sprints* de implementación a las fases de entrega del proyecto establecidas por la UNIR, de modo que pudiéramos ir adecuándonos a las exigencias e indicaciones de nuestros profesores mientras manteníamos un buen ritmo de trabajo.

Puesto que el equipo está formado por tres miembros con distinta disponibilidad de tiempo y preferencias, decidimos distribuir las tareas teniendo en cuenta estos dos aspectos. Los roles están diferenciados, aunque el sistema de trabajo sea colaborativo:

- Laura: desarrollo del backend, bases de datos y lógica de negocio.
- Patricia: desarrollo de la guía de estilo, representación del grupo y desarrollo del sitio web público.
- Santiago: desarrollo del dashboard de administración.

Aunque se haya establecido un reparto de roles, las decisiones son siempre compartidas y los avances se abordan y acuerdan en grupo.

Para la fase de pruebas, la metodología que hemos establecido, en parte por nuestras carencias de dominio como estudiantes, ha sido la realización de pruebas de los *endpoints* con Postman, acompañadas de pruebas funcionales de navegación y uso (en las que comprobamos la usabilidad del sitio y el correcto funcionamiento de los componentes), y de solicitudes de feedback a usuarios externos. Hemos realizado también pruebas de integración de todo el sistema, para verificar el funcionamiento del envío de correos y el uso de la pasarela de pago (en una primera fase, simulada con Mailtrap).

5. DESARROLLO DEL PROYECTO

5.1. Introducción: estudio y análisis de la aplicación

La aplicación debía cumplir con unas funciones indispensables que facilitasen la comunicación entre el cliente y el tatuador. Para ello, nos centramos en diseñar una aplicación minimalista y fácil de usar.

Para la parte del cliente, vimos indispensable una página web con un formulario principal. En dicho formulario, el cliente indica el servicio que desea contratar de forma detallada. Con esto, podríamos almacenar en la base de datos los servicios de los clientes y, así, hacerlos llegar a los tatuadores en la interfaz de administrador o trabajador.

Otro punto importante era la propuesta de la cita, es decir, darle al cliente unas opciones de días y horas donde pudiera programar su cita. De esta forma, el cliente podría seleccionar una fecha y un horario que le resultasen convenientes. Esto también se almacenaría en la base de datos, mejorando así la organización de la agenda del tatuador. Añadimos la opción de bloquear esa fecha y hora hasta que el servicio se confirma definitivamente para evitar futuros problemas de solapamiento de citas.

De la misma forma que ofrecíamos al cliente la posibilidad de solicitar citas, también debía existir la opción de modificarlas (en cuanto a la fecha) y rechazarlas, ya que siempre pueden surgir inconvenientes de horarios o darse el caso de que el cliente finalmente decida no proceder con el servicio.

Para evitar problemas de compromiso por parte del cliente y ausencias en las citas, vimos necesario elaborar un sistema de fianzas. De este modo, el cliente se comprometería a asistir y, en caso de que no lo hiciera, el tatuador no perdería tanto dinero por haber bloqueado esa fecha. Así, las citas confirmadas con una fecha reservada estarían respaldadas por una parte del valor del servicio.

Otro punto importante para el cliente era dotarle de la información del presupuesto. Con el conocimiento de este, el cliente tendría la opción de aceptarlo o rechazarlo. Para ello, vimos el envío de emails como la mejor opción, ya que es la metodología que más se usa hoy en día

para el envío de presupuestos y precios. Además, resulta más profesional que enviarlo por WhatsApp, ya que le podemos dar un formato más personalizado.

Por otro lado, tenemos la interfaz del tatuador. Esta interfaz tendría como objetivo principal mostrar la agenda del profesional; es decir, indicarle cuáles son sus citas cercanas en el día o la semana. Además de esto, era indispensable la revisión de nuevas solicitudes y el envío de sus presupuestos. De esta forma, la aplicación tendría el flujo completo de las citas: creación, revisión, propuesta de precio, realización y finalización.

Para la parte de la finalización de la cita, vimos necesario un sistema de facturación, ya que, en función de la petición del cliente, el establecimiento debía disponer de la posibilidad de entregarle una factura conforme a la legalidad. Por lo tanto, el proceso de finalización debería tener un apartado para crear facturas automáticamente con los datos de la empresa, los del cliente y los del servicio realizado.

En los siguientes subapartados de esta sección detallaremos el proceso de creación e implementación de cada uno de los elementos del proyecto, profundizando en las tecnologías que hemos seleccionado y las decisiones tomadas a medida que avanzábamos.

5.2. Aspecto visual del proyecto

5.2.1. Imagen de marca y guía de estilo:

Antes de proceder con la creación de código en sí, se decidió llevar a cabo una fase de diseño visual para darle al estudio ficticio INK&CO, que es el cliente final de nuestro equipo de desarrollo, una identidad de marca sobre la que construir tanto el sitio web como el dashboard, de modo que ambos tuviesen cohesión en su aspecto visual y los elementos gráficos.

Con este fin, se recurrió a Figma para elaborar una guía de estilo a partir de sitios web aspiracionales cuya propuesta visual nos había resultado atractiva. Este documento serviría además como apoyo para simplificar el proceso de diseño de ambas partes del frontend, garantizando la cohesión y agilizando el proceso de desarrollo posterior.

En esta guía, se optó por una combinación de dos tipografías, Poppins y Figtree, buscando una imagen moderna, legible y elegante. La tipografía Poppins resulta ideal para encabezados y títulos por su fuerza visual y la armonía de sus líneas, mientras que la tipografía Figtree es muy legible en párrafos más largos, lo cual mejora la visualización y accesibilidad del diseño.

La paleta de colores se definió con el apoyo del sitio web colors.com, que permite verificar de forma sencilla la visibilidad y el grado de contraste de los colores seleccionados para la mayoría de las discapacidades visuales que podría presentar un usuario del sitio web.



Figura 3. Verificación de visibilidad la paleta de color en Colors.com

Se optó por seleccionar dos tonos verdosos y dos rojizos, acompañados de otros colores neutros como el blanco y el gris oscuro. Los tonos más vivos atraen la mirada, se asemejan a colores vibrantes de algunos tatuajes de gran belleza visual y encajan por lo tanto en la imagen que queríamos darle al estudio.

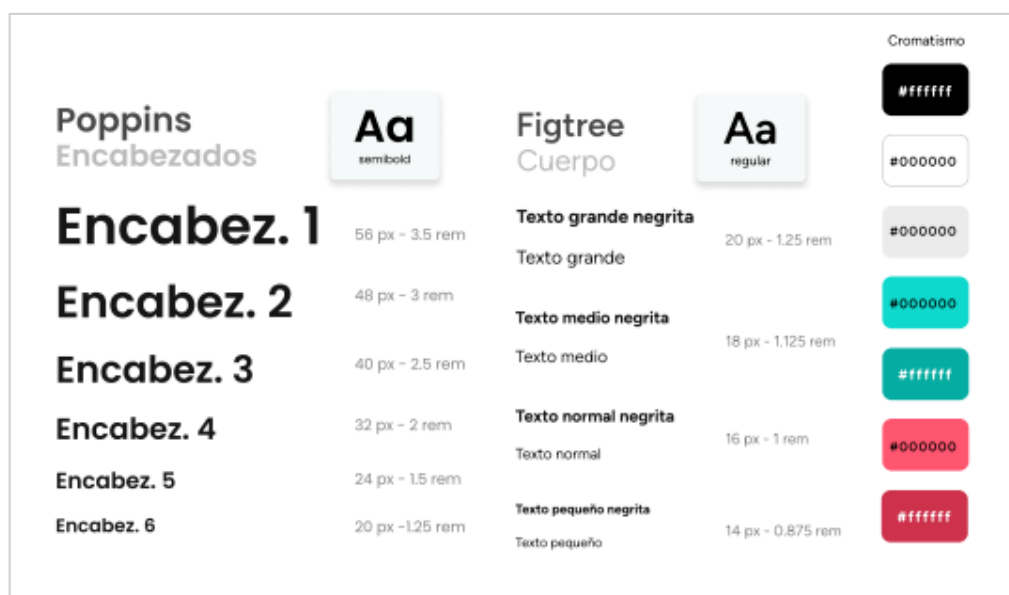


Figura 4. Captura de sección de tipografía y color en la guía de estilo.

Esta guía también define aspectos concretos necesarios para las funcionalidades establecidas en la fase de análisis, como recuadros de input o botones para los formularios, o las vistas del dashboard.

Como complemento, también se diseñó un logotipo para el estudio, en la herramienta gratuita logo.com, que representa un recipiente de tinta junto al nombre del establecimiento, con un aspecto de ilustración tipo vectorial, minimalista y con relleno de color liso, coherente con lo establecido en la guía de estilo y con la imagen deseada para el estudio de tatuaje. Se decidió usar un formato .svg para facilitar el escalado y reutilización en el sitio web, además de para reducir el “peso” de los elementos del sitio en su posible despliegue.



Figura 5. Logotipo diseñado para el estudio INK&CO.

Una vez conceptualizados todos estos elementos, la paleta y las tipografías se implementaron en el desarrollo del código como variables CSS globales para permitir un uso centralizado de los estilos y evitar incoherencias en las distintas páginas y componentes del frontend.

5.2.2. Prototipado del proyecto y flujo de uso

Durante el proceso de diseño de la interfaz de usuario y la experiencia de usuario, hemos hecho un recorrido evolutivo desde el punto de partida más analógico: un boceto a mano alzada en papel. Se realizaron esquemas y bocetos de los elementos del sitio web, sus diferentes vistas, o la disposición de los elementos para ir depurando una estructura básica sobre la que seguir avanzando en fases posteriores.

Estos diseños primigenios se trasladaron más tarde en forma de wireflows (ANEXOS 2 y 3) y mockups (ANEXOS 4 a 9) elaborados con dos plantillas gratuitas muy versátiles de Figma.

Para el wireflow, se llevó a cabo una propuesta que abarcaba todas las posibles páginas que visitaría un usuario del sitio web, y que mostraba qué elementos estaban comunicados entre sí o cómo fluía la visita del usuario a medida que este interactuaba con distintos botones o funcionalidades. Este punto fue crucial para poder visualizar la dimensión de ambas partes del frontend, y decidir si se nos había escapado algo durante la fase de análisis inicial.

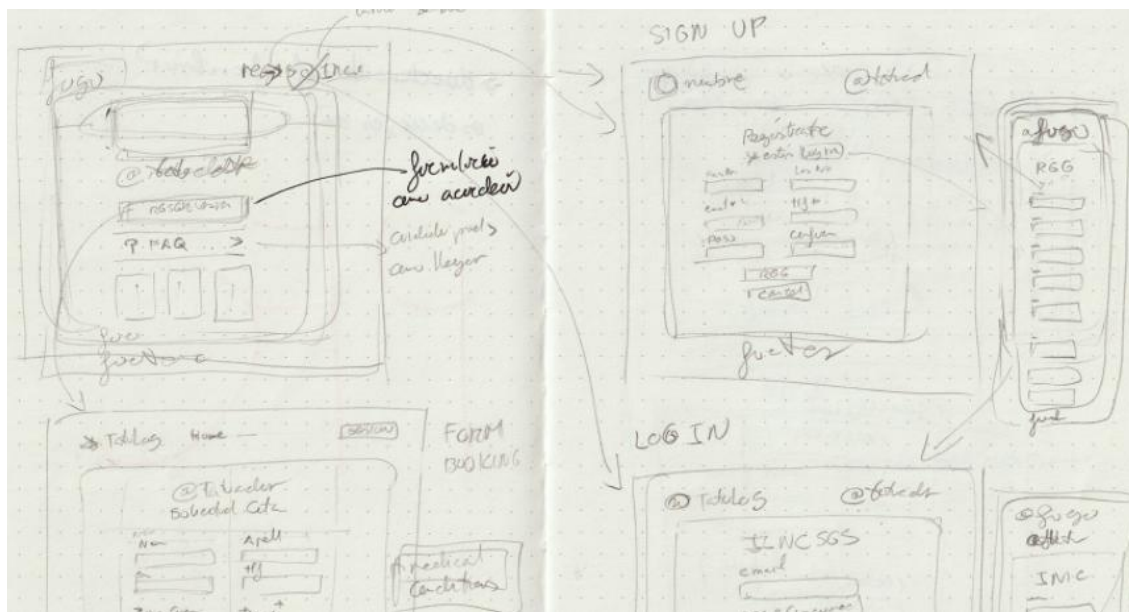


Figura 6. Fragmento de bocetos en papel para el diseño del sitio web.



Figura 7. Fragmento del wireflow elaborado para el sitio web.

Con posterioridad, se elaboraron los mockups de las vistas del frontend en Figma, a modo de prueba de la solidez del concepto y las ideas para la organización visual de los elementos. Estos mockups, a su vez, se trasladaron a prototipos simples en formato .html, con el propósito de poder emplearlos como punto de partida en el desarrollo en Angular.



Figura 8. Captura del mockup de Figma (versión de escritorio) del sitio.

5.3. Desarrollo del frontend (sitio web del estudio)

El frontend del proyecto se ha desarrollado con el framework Angular, puesto que se trata de uno de los frameworks incluidos en el currículo del ciclo formativo y su uso representaba una excelente oportunidad para familiarizarnos con su funcionamiento, dada nuestra escasa experiencia con este.

Se tomó la decisión de crear un solo proyecto en el que se alojaría tanto el sitio web orientado a los clientes finales del estudio como el dashboard o panel de administración creado para los propietarios y trabajadores del estudio INK&CO. Las razones que fundamentaron la decisión fueron, en primer lugar, la conciencia de que, como estudiantes, trabajar por separado en un proyecto de esta magnitud podría llevarnos a desarrollar por duplicado ciertos aspectos que podrían ser comunes, y que seguramente sería más difícil para nosotros coordinar dos proyectos paralelos para ambas partes del frontend con el desarrollo del backend y el mantenimiento de la base de datos.

5.3.1. Arquitectura y organización de los componentes del sitio web

Se ha procurado crear una estructura de carpetas que modulariza el frontend, de modo que los dos miembros del equipo que hemos desarrollado cada “extremidad”, por así decir, de este, pudiéramos ir avanzando y no acabásemos mezclando o confundiendo elementos que correspondiesen a la otra vertiente. Ambos compartimos algunos elementos (como es el caso del servicio denominado AppointmentService, que se utiliza para todo lo relativo a las citas) y disponemos también de compartimentos separados para aquello que no conviene que sea común.

Esto ha supuesto un desafío: el ritmo de avance del curso nos ha forzado a ir implementando aspectos sobre los que aún no teníamos conocimiento o dominio suficiente, por lo que hemos tenido que buscar información, tutoriales y apoyo de inteligencias artificiales para saber cómo proceder. Aunque en algunos momentos esto ha generado indecisión y dudas acerca de si estábamos tomando buenas decisiones, es cierto que ha resultado muy instructivo al ir encontrando obstáculos y problemas que debíamos resolver para avanzar e ir creando una aplicación mínimamente funcional.

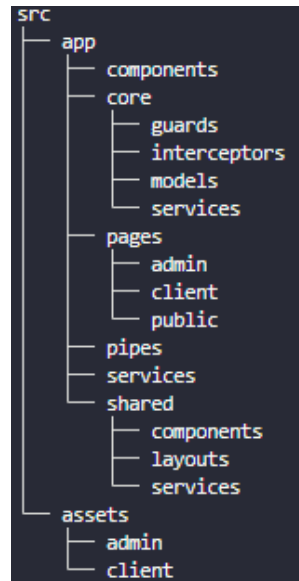


Figura 9. Estructura de carpetas del proyecto.

El sitio web en sí mismo está concebido como una SPA (Single Page Application) cuyo principal objetivo es la conversión de los visitantes. La página principal sirve como “contenedor” para los elementos principales: el formulario y un recuadro donde el cliente puede modificar la fecha de una cita confirmada, o cancelarla. A continuación, se profundiza un poco más en el funcionamiento de ambos:

Formulario para solicitar citas: el usuario dispone de un botón que despliega un formulario en el que puede especificar, con gran precisión, qué tipo de servicio desea. Puede incluir los datos personales indispensables para que el estudio pueda enviarle comunicaciones y escoger entre parámetros comprensibles y relevantes, como la zona del cuerpo, el estilo o el tamaño del diseño que desea tatuarse, retocar o incluso eliminar. Estos parámetros se establecieron durante la fase de análisis, y se basaron en los servicios concretos y estilos que ofrece el estudio a sus clientes. Además, el usuario puede indicar si desea una factura de su servicio y adjuntar imágenes de referencia para que el profesional asignado por el sistema pueda realizar un diseño acorde a las expectativas de su cliente.

El desarrollo de este formulario se ha ido complejizando con el paso del tiempo: comenzó siendo un armazón con funcionalidad básica (desplegables, botones, etc.) para pasar a versiones cada vez más completas en las que se ha implementado la verificación del formato de los datos esperados, la limitación de formatos y tamaños de imagen permitidos como adjunto, o la exigencia de que el usuario marque la casilla de los términos y condiciones para poder enviar su solicitud.

Nombre*

Indica tu nombre...

El nombre es obligatorio.

Tamaño* ⓘ

En caso de duda entre dos tamaños, elige el más grande.

Elige un tamaño (aproximado)...

Estilo* ⓘ

¿Clásico, minimalista...?

Figura 10. Ejemplos de validaciones y pop-up de ayuda del formulario.

Uno de los aspectos más complejos fue el apartado de selección de fecha y hora, que aparece dinámicamente cuando se han seleccionado los parámetros base del servicio. Esto comenzó siendo un mockup que simulaba el funcionamiento para ir realizando pruebas, para posteriormente pasar a ser un elemento funcional en coordinación con la compañera encargada del backend y la base de datos.

Esta implementación ha exigido el uso de Reactive Forms, ya que es algo que hemos trabajado brevemente en las clases y parecía la mejor opción para el funcionamiento que buscábamos. Además, como el formulario debe enviar una serie de datos para que se realice el cálculo de la duración de una cita, y poder ofrecer huecos de esa duración en el calendario al cliente antes de que complete la solicitud, ha sido necesario el uso de servicios inyectables con HttpClient.

Esto ha supuesto un desafío por la falta de experiencia, y se ha tenido que desarrollar el código con muchos cambios por el camino a medida que observábamos si el comportamiento era el esperado, encontrábamos errores de funcionamiento en el frontend o el backend, o nos dábamos cuenta de deficiencias de concepto.

La imagen muestra un selector de fecha y hora para un formulario de reserva. En la parte superior, se indica el mes "FEBRERO DE 2026". Debajo, hay una fila de días de la semana (L, M, X, J, V, S, D) y una fila de números de días (9, 10, 11, 12, 13, 14, 15). El día 12 de febrero está seleccionado y resaltado en azul. Los días 14 y 15 están marcados con una 'x' roja, indicando que no están disponibles. Debajo de la selección de fecha, se muestra la sección "Horas disponibles:" con una fila de botones de hora: 10:00, 10:30, 11:00, 11:30, 12:00, 12:30, 13:00, 13:30, 14:00, 14:30, 15:00, 15:30, 16:00, 16:30, 17:00, 17:30, 18:00, 18:30, 19:00. En la parte inferior, hay dos botones: "BORRAR" y "CANCELAR".

Figura 11. Selector de fecha y hora del formulario de reserva de citas.

Modificador de citas: además del formulario, el usuario que ya ha solicitado citas previamente puede utilizar una sección de cambio de cita para modificar la fecha y el horario de esta, o cancelarla en caso de que no pueda asistir.

Este componente también ha ido cambiando durante el desarrollo. Su primera versión empleaba el identificador de las citas (un id sencillo establecido como auto-increment) para recuperar la información de la base de datos y modificarla.

Una vez comprobado que la conexión backend <-> frontend funcionaba, se procedió a implementar que esta recuperación se llevase a cabo mediante un localizador único para cada cita, que se genera aleatoriamente en la base de datos en el momento de la solicitud, y que el cliente recibe en el correo electrónico de confirmación.

De este modo, el cliente debe introducir este localizador (algo que tiene) y el correo electrónico con el que solicitó la cita (algo que es) para poder acceder a los datos y modificar la fecha, con lo que garantizamos un uso más seguro de esta funcionalidad, que también es más amigable con el usuario.

El formulario tiene un fondo oscuro con los campos de entrada en blanco. El título 'GESTIONA TU CITA' está en el centro en letras blancas. Debajo, una instrucción en blanco indica que se debe introducir el localizador y el correo electrónico. Los campos de entrada están etiquetados con 'Localizador (Referencia)' y 'Email asociado' en rojo. El primer campo contiene el texto 'EJ: A5B6F1C2' y el segundo 'TU@EMAIL.COM'. En la parte inferior, hay dos botones: 'VOLVER' y 'BUSCAR'.

Figura 12. Recuadro para modificación o cancelación de cita.

Por último, el sitio también dispone de un acordeón de preguntas frecuentes que aclara dudas y proporciona información útil sobre el proceso de reserva, la cancelación o los parámetros de los servicios que ofrece el estudio.

A modo de contenido extra, para dotar al sitio de un aspecto más real y completo, se ha añadido la página de Términos y Condiciones (accesible desde el footer y el formulario) y una página de contacto que permite enviar un mensaje a la bandeja de entrada del correo del estudio.

En lo que respecta al aspecto más técnico de este frontend, se ha utilizado un enfoque mixto de maquetación, con una combinación de Bootstrap para la mayor parte del contenido y la estructura básica del sitio, y elementos de CSS personalizados, Grid y Flexbox para conseguir un comportamiento y disposición de elementos más refinado y coherente con los mockups que se habían creado en las fases previas. También se han incluido media queries con la idea de mejorar la visualización de todos los elementos (como los botones) en pantallas de teléfono móvil estándar (se ha tomado un Samsung S8 como referencia) y pantallas de escritorio.

```
<div class="col-md-6">  
  <label class="form-label d-block">¿Color o B/N?</label>  
  <div class="d-flex gap-3 mt-2 radio-align-fix">
```

Figura 13. Ejemplo de uso de Bootstrap en la página de inicio del sitio web.

```
/* =====  
RESPONSIVE - MÓVIL VERTICAL  
===== */  
@media (max-width: 480px) {  
  
  /* ==== SECCIÓN HERO ==== */  
  #hero {  
    height: 55vh;  
    min-height: 400px;  
  }  
  
  .hero-text {  
    text-align: center;  
    padding: 1rem 1rem;  
    padding-top: 4em;  
  }  
}
```

Figura 14. Ejemplo de uso de media queries en el sitio web.

Durante todo el proceso se han ido realizando pruebas de visualización del sitio web en distintos navegadores y pantallas, además de pruebas de usabilidad y accesibilidad de los elementos. También se ha contado con la colaboración de usuarios externos que han aportado feedback sobre la fluidez de uso, el aspecto visual o el tamaño y disposición de los elementos.

Y como ya se ha convertido en la tónica habitual de nuestro proyecto, todos los avances y los planteamientos para cada paso del desarrollo se han ido comunicando a los compañeros, de modo que siempre contásemos con un consenso antes de implementar algún cambio, avance o nueva funcionalidad, y también para corroborar si cada nueva iteración de este frontend funcionaba de la forma esperada o si surgían errores al realizar pruebas en nuestros equipos.

La fase final de desarrollo de esta parte del proyecto ha consistido en la revisión y limpieza del código, para eliminar métodos deprecados, añadir comentarios útiles para el mantenimiento y reorganizar estilos, entre otros aspectos. Además, se ha procedido a revisar la visualización de cada página y elemento en distintos tamaños de pantalla, añadiendo ajustes donde fuera necesario; así como a realizar pruebas de accesibilidad con Lighthouse y Axe, para corregir posibles problemas de contraste o navegación. Los problemas encontrados fueron mínimos, relacionados principalmente con grados de contraste que fueron fáciles de corregir.

5.4. Arquitectura del backend

5.4.1. Diseño y modelado de la base de datos

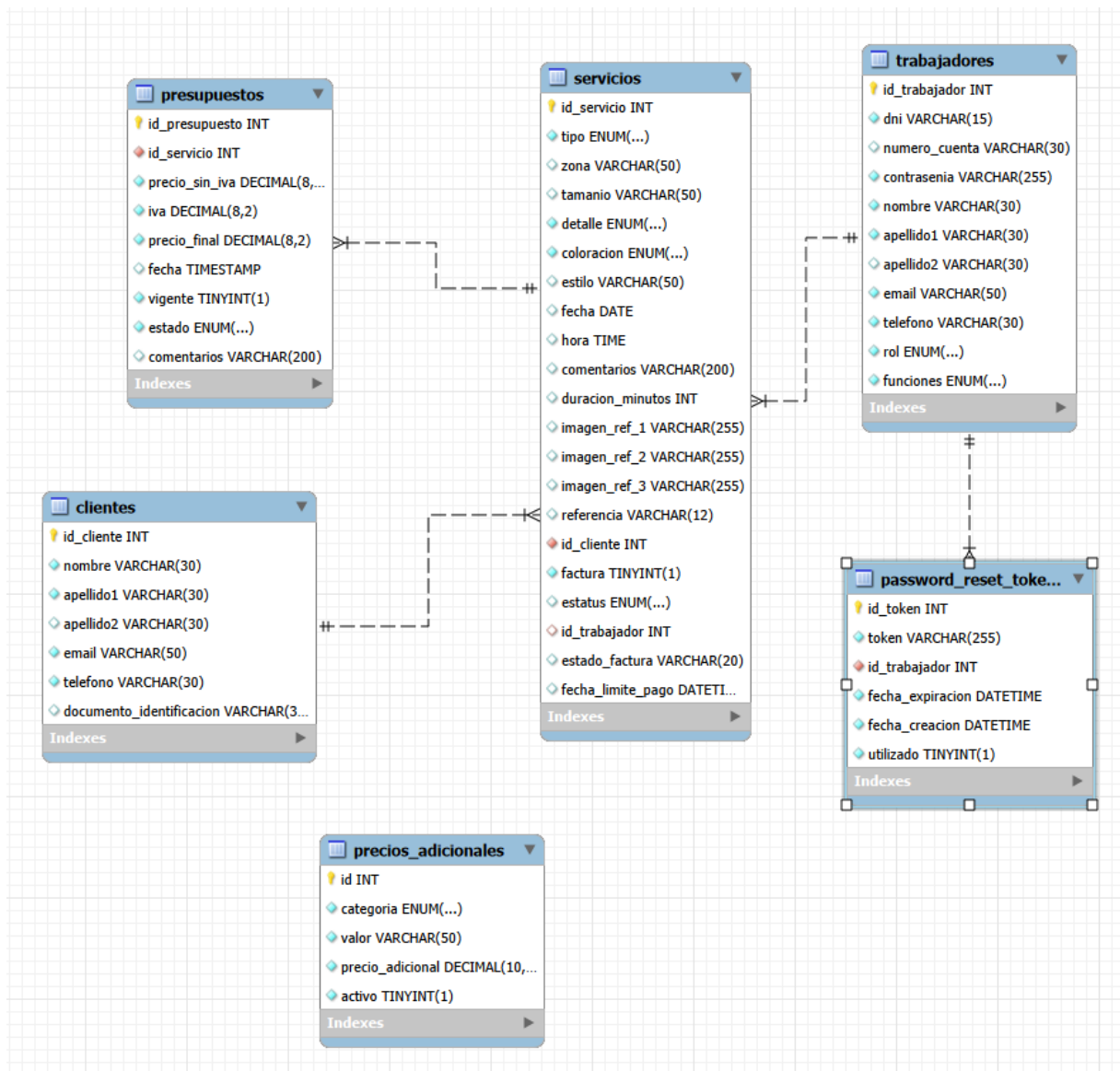


Figura 15. Diagrama entidad-relación generado por ingeniería inversa desde la base de datos MySQL.

5.4.1.1 Esquema de la base de datos

El diseño de la base de datos de TatuSys se fundamenta en un modelo relacional que refleja la estructura operativa de un estudio de tatuajes. El esquema ha sido concebido para gestionar de forma eficiente las relaciones entre clientes, trabajadores, servicios y presupuestos, estableciendo un flujo de datos coherente que soporta las operaciones del negocio.

La estructura de la base de datos se ha organizado siguiendo una arquitectura jerárquica de tres niveles de dependencias, lo que garantiza la integridad referencial y facilita el mantenimiento:

Nivel 0 - Entidades independientes:

- **Clientes:** almacena la información personal y de contacto de los usuarios del estudio.
- **Trabajadores:** contiene los datos de empleados, incluyendo credenciales de acceso y permisos.
- **Precios adicionales:** define los costes asociados a diferentes características de los servicios.

Nivel 1 - Entidades dependientes:

- **Servicios:** núcleo del sistema que registra las citas y trabajos, con dependencias hacia clientes y trabajadores.
- **Password reset tokens:** gestiona los tokens de recuperación de contraseña vinculados a trabajadores, registrando su fecha de creación, fecha de expiración y si han sido utilizados, garantizando un flujo seguro de restablecimiento de credenciales.

Nivel 2 - Entidades de gestión:

- **Presupuestos:** maneja la información económica vinculada a cada servicio.

Características:

- **Normalización:** el esquema sigue las formas normales hasta 3FN, eliminando redundancias y garantizando la consistencia de los datos.
- **Flexibilidad:** el uso estratégico de campos ENUM proporciona valores controlados para categorías como tipos de servicios, zonas corporales y estilos de tatuaje, manteniendo la integridad semántica.
- **Seguridad:** la tabla password_reset_tokens implementa un mecanismo de tokens de un solo uso con expiración temporal, evitando reutilizaciones y accesos no autorizados al sistema de recuperación de contraseñas.

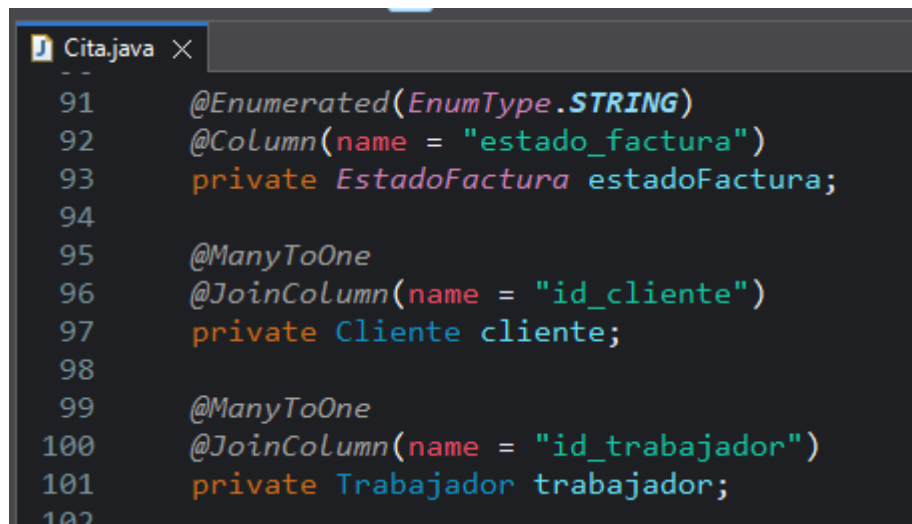
- **Escalabilidad:** la estructura modular permite la incorporación de nuevas funcionalidades sin comprometer la arquitectura existente.
- **Optimización:** se han implementado índices estratégicos para mejorar el rendimiento en consultas frecuentes, especialmente en búsquedas por documento de identificación y combinaciones únicas de identificación de clientes.

5.4.1.2 Entidades JPA y mapeo Objeto-Relacional

La implementación del mapeo objeto-relacional en TatuSys utiliza JPA (Java Persistence API) con Hibernate como proveedor de persistencia. La estrategia de mapeo adoptada se basa en los siguientes principios:

Mapeo directo: cada tabla de la base de datos corresponde a una entidad JPA, manteniendo una correspondencia directa entre el modelo relacional y el modelo de objetos.

Gestión de relaciones: las relaciones entre entidades se implementan mediante anotaciones JPA:



```
Cita.java X
91     @Enumerated(EnumType.STRING)
92     @Column(name = "estado_factura")
93     private EstadoFactura estadoFactura;
94
95     @ManyToOne
96     @JoinColumn(name = "id_cliente")
97     private Cliente cliente;
98
99     @ManyToOne
100    @JoinColumn(name = "id_trabajador")
101    private Trabajador trabajador;
102
```

Figura 16. Ejemplo de mapeo en la entidad Cita.

Configuraciones específicas

Estrategia de claves primarias: se utiliza `@GeneratedValue` con estrategia `IDENTITY` para el autoincremento de claves primarias, aprovechando las capacidades nativas de MySQL.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id_servicio")
private int idCita;
```

Figura 17. Ejemplo de mapeo `@GeneratedValue` en la entidad Cita.

Mapeo de enumerados: los campos ENUM de la base de datos se mapean utilizando `@Enumerated` con `EnumType.STRING`, manteniendo la legibilidad y facilitando el mantenimiento.

```
@Column(nullable = false)
@Enumerated(EnumType.STRING)
private Tamano tamano;

@Column(nullable = false)
@Enumerated(EnumType.STRING)
private Detalle detalle;

@Column(nullable = false)
@Enumerated(EnumType.STRING)
private Coloracion coloracion;

@Column(nullable = false)
@Enumerated(EnumType.STRING)
private Estilo estilo;
```

Figura 18. Ejemplo de mapeo `@Enumerated` en la entidad Cita.

Gestión de fechas: los campos temporales se manejan con anotaciones `@Temporal`, especificando el tipo apropiado (`DATE`, `TIME`, `TIMESTAMP`) según el contexto de uso.

```
@Temporal(TemporalType.DATE)
private LocalDate fecha;

@Temporal(TemporalType.TIME)
private LocalTime hora;
```

Figura 19. Ejemplo de mapeo @Temporal en la entidad Cita.

5.4.2 Implementación de la API REST con Spring Boot

5.4.2.1 Arquitectura de capas del sistema

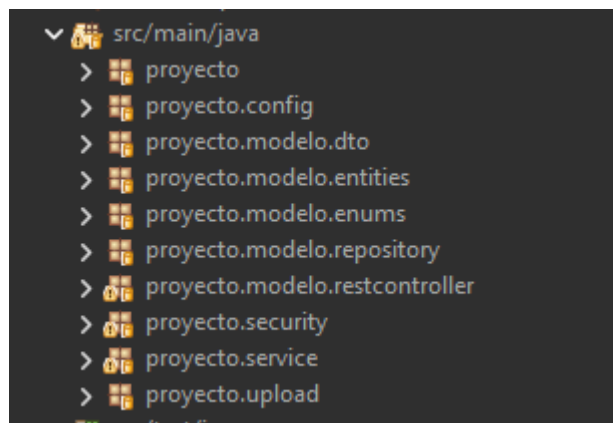


Figura 20. Estructura de paquetes.

La implementación de TatuSys sigue una arquitectura de capas claramente definida, organizando el código en paquetes específicos que separan responsabilidades y facilitan el mantenimiento:

Capa de presentación:

- `proyecto.modelo.restcontroller` - Controladores REST que exponen los endpoints de la API.

Capa de lógica de negocio:

- `proyecto.service` - Interfaces y servicios que implementan la lógica central de la aplicación.

Capa de acceso a datos:

- `proyecto.modelo.repository` - Repositorios JPA para interactuar con la base de datos.
- `proyecto.modelo.entities` - Entidades JPA que mapean las tablas de base de datos.

Capas de apoyo:

- `proyecto.modelo.dto` - Objetos de transferencia de datos para comunicación externa.
- `proyecto.security` - Configuración de seguridad, autenticación y autorización.
- `proyecto.modelo.enums` - Enumerados que definen valores controlados del dominio.
- `Proyecto.modelo.upload` - Gestión de carga y almacenamiento de archivos en las capas de apoyo.

Principios arquitectónicos:

- **Separación de responsabilidades:** cada capa tiene una función específica y bien delimitada, evitando el acoplamiento directo entre capas no adyacentes.
- **Inversión de dependencias:** los servicios se definen mediante interfaces, permitiendo flexibilidad en las implementaciones y facilitando las pruebas unitarias.
- **Patrón MVC adaptado:** la arquitectura sigue una variante del patrón Modelo-Vista-Controlador adaptada para APIs REST, donde los controladores manejan las peticiones HTTP y delegan la lógica de negocio a los servicios.

5.4.2.2 Capa de controladores REST

Los controladores REST implementan la anotación `@RestController` y `@RequestMapping` para definir los endpoints base. La aplicación cuenta con controladores especializados:

CitaRestController - Controla todas las operaciones relacionadas con citas y servicios, incluyendo creación, modificación, eliminación, búsquedas avanzadas, gestión de estados, asignación de trabajadores y cálculo de disponibilidad.

ClienteRestController - Maneja la gestión completa de clientes, incluyendo registro, actualización de datos personales, búsquedas por diferentes criterios y gestión del historial de servicios.

LimpiezaController - Implementa tareas de mantenimiento del sistema, incluyendo limpieza de datos temporales, optimización de base de datos y tareas programadas de mantenimiento.

PreciosRestController - Administra el sistema de precios adicionales y tarifas del estudio, permitiendo la configuración dinámica de precios por categorías, zonas, estilos y otros factores que influyen en el coste final.

PresupuestoRestController - Controla la generación, modificación y gestión de presupuestos, incluyendo cálculos automáticos, aplicación de descuentos, gestión de estados de presupuestos y generación de documentos.

TrabajadorRestController - Maneja la administración de trabajadores y recursos administrativos, incluyendo alta, baja, modificación de datos, gestión de roles y permisos, y visualización de citas asignadas.

UploadRestController - Gestiona la carga y administración de archivos del sistema, incluyendo imágenes de referencia, documentos adjuntos, validación de formatos y almacenamiento seguro de archivos.

MigrationController - Controlador temporal de desarrollo que gestiona la migración de datos del sistema, específicamente la conversión de contraseñas de texto plano a formato encriptado durante las actualizaciones del sistema.

Manejo de peticiones HTTP

Mapeo de operaciones CRUD: los controladores implementan el mapeo completo de operaciones HTTP mediante anotaciones específicas:

- `@GetMapping` para operaciones de consulta.

```
@GetMapping("/trabajadores/{trabajadorId}/citas") //SEGURIDAD OKEI
public ResponseEntity<?> obtenerCitasDelTrabajador(@PathVariable int trabajadorId) {
```

- `@PostMapping` para creación de recursos.

```
@PostMapping("/trabajador-alta")
public ResponseEntity<?> altaTrabajador(@RequestBody Trabajador trabajador) {
    try {
```

- `@PutMapping` para actualizaciones completas.

```
@PutMapping("/trabajadores/{trabajadorId}") //SEGURIDAD OK
public ResponseEntity<?> actualizarTrabajador(@PathVariable int trabajadorId, @RequestBody Trabajador trabajador) {
```

- `@DeleteMapping` para eliminación de recursos.

```
@DeleteMapping("/trabajadores/{trabajadorId}") //SEGURIDAD OK
public ResponseEntity<?> eliminarTrabajador(@PathVariable int trabajadorId) {
    int resultado = trabajadorService.eliminarTrabajador(trabajadorId);
```

Gestión de parámetros: se utilizan múltiples estrategias para capturar parámetros:

- `@PathVariable` para variables en la URL.
- `@RequestParam` para parámetros de consulta.
- `@RequestBody` para datos JSON en el cuerpo de la petición.
- `@RequestPart` para datos multipart (archivos).

Manejo de archivos

La aplicación implementa subida de archivos mediante `MultipartFile`, permitiendo a los usuarios adjuntar imágenes de referencia para los servicios.

```
// --- MÉTODO AUXILIAR PARA GUARDAR EN DISCO ---
private String guardarArchivo(MultipartFile archivo) throws IOException {
```

5.4.2.3 Capa de servicios y lógica de negocio

La capa de servicios implementa el patrón de interfaz-implementación donde, por ejemplo, `CitaService` define contratos que son implementados por la clase específica `CitaServiceImplJpaMy8`.

Algunos ejemplos específicos de lógica de negocio de especial relevancia son:

Cálculo automático de presupuestos: el sistema implementa un motor de cálculo que determina precios basándose en múltiples factores como tipo de servicio, zona corporal, tamaño, nivel de detalle, coloración y estilo artístico.

Gestión de disponibilidad: se implementa un algoritmo que calcula huecos disponibles en la agenda de trabajadores, considerando la duración estimada de los servicios y los horarios comerciales.

Asignación automática de trabajadores: el sistema incluye lógica para seleccionar automáticamente un trabajador disponible basándose en el tipo de servicio requerido. Actualmente distingue entre dos funciones: si el servicio es de tipo `ELIMINACION` asigna un trabajador con función `ELIMINACION`, y para el resto de los tipos (tatuaje, cover, retoque) asigna un trabajador con función `CREACION`. Entre los candidatos disponibles con la función requerida, la selección se realiza de forma aleatoria. El código incluye una zona preparada y comentada para incorporar en el futuro una lógica más específica basada en el estilo artístico (japonés, fineline, etc.), que podrá activarse cuando el enum `Funciones` disponga de más valores especializados.

Sistema de referencias: se genera automáticamente un código de referencia único para cada cita, permitiendo a los clientes acceder a sus servicios sin necesidad de autenticación completa.

La aplicación maneja múltiples estados tanto para citas (`PENDIENTE`, `CONFIRMADO`, `FINALIZADO`) como para presupuestos (`PENDIENTE`, `GENERADO`, `ACEPTADO`, `RECHAZADO`), implementando transiciones controladas entre estados.

```
2
3 public enum Estado {
4     PENDIENTE, ACEPTADO, RECHAZADO, GENERADO, FINALIZADO
5 }
6

public enum Estatus {
    PENDIENTE, CONFIRMADO, FINALIZADO
}
```

Figura 21. Ejemplo de enums de los estados.

5.4.2.4 Capa de repositorios y acceso a datos

La aplicación utiliza Spring Data JPA con repositorios que extienden JpaRepository, proporcionando operaciones CRUD automáticas y permitiendo definir consultas personalizadas.

Se implementan métodos de consulta personalizados como:

- findByDniIgnoreCase() para búsquedas insensibles a mayúsculas.
- findByRol() para filtrado por roles de trabajador.
- findByReferencia() para búsqueda por código de referencia.

```
public interface TrabajadorRepository extends JpaRepository<Trabajador, Integer>{

    /*Optional: Puede contener un valor o estar vacío*/
    Optional<Trabajador> findByEmail(String email);

    Optional<Trabajador> findByDniIgnoreCase(String documento);

    List<Trabajador> findByFunciones(Funciones funcionRequerida);

    List<Trabajador> findByRol(Rol rol);
}
```

Figura 22. Ejemplo de repositorio con herencia de JpaRepository.

5.4.2.5 Sistema de seguridad y autenticación

El sistema de seguridad de TatuSys implementa una arquitectura completa basada en componentes especializados organizados en el paquete proyecto.security, proporcionando autenticación, autorización y gestión segura de sesiones.

Componentes del sistema de seguridad

SecurityConfig - Configuración principal de seguridad que define las reglas de autorización, configuración CORS, manejo de sesiones stateless y la cadena de filtros de seguridad.

JwtAuthenticationFilter - Filtro personalizado que intercepta las peticiones HTTP para validar tokens JWT, extrayendo y verificando la autenticación antes de permitir el acceso a los recursos protegidos.

JwtService - Servicio especializado en la gestión de tokens JWT, incluyendo generación, validación, extracción de claims, y manejo del ciclo de vida de los tokens de acceso.

AuthenticationService - Servicio central de autenticación que coordina el proceso de login, validación de credenciales y generación de respuestas de autenticación.

SecurityUtils - Utilidad centralizada que proporciona métodos reutilizables para verificación de permisos, validación de roles y control de acceso contextual a recursos específicos.

UserDataServiceImpl - Implementación del servicio de datos de usuario que gestiona la carga y validación de información de usuarios para el proceso de autenticación.

Modelos de Datos de Seguridad

LoginRequest - Modelo de datos que encapsula las credenciales de usuario recibidas en las peticiones de autenticación, incluyendo validaciones de formato y contenido.

LoginResponse - Estructura de respuesta estandarizada para operaciones de autenticación, conteniendo tokens, información de usuario y datos de sesión.

ErrorResponse - Modelo unificado para respuestas de error del sistema de seguridad, proporcionando información estructurada sobre fallos de autenticación y autorización.

Configuración de Seguridad

Configuración de Seguridad

Autenticación JWT: el sistema implementa autenticación stateless mediante tokens JWT, eliminando la necesidad de mantener sesiones en el servidor y mejorando la escalabilidad de la aplicación.

Autorización por roles: se definen dos roles principales:

- **ADMIN** - Acceso completo a todas las funcionalidades del sistema.
- **TRABAJADOR** - Acceso restringido a recursos propios y funcionalidades específicas.

Control de acceso granular: además de los roles básicos, el sistema implementa validaciones contextuales que verifican que los trabajadores solo puedan acceder a sus propios recursos, como citas asignadas e información personal.

Configuración CORS: el sistema incluye configuración CORS específica para permitir peticiones desde frontends de desarrollo, con soporte completo para métodos HTTP y headers personalizados.

Encriptación de contraseñas: las contraseñas se encriptan utilizando BCrypt antes de almacenarse en la base de datos, garantizando la seguridad de las credenciales.

Recuperación de contraseñas: el sistema implementa un mecanismo seguro de recuperación de contraseñas gestionado por PasswordResetTokenService, que cubre el ciclo de vida completo del token: creación vinculada al trabajador, validación de vigencia y expiración, marcado como utilizado para evitar reutilizaciones, limpieza automática de tokens expirados y eliminación de tokens asociados a un trabajador concreto. Adicionalmente expone un método de auditoría para contabilizar tokens creados desde una fecha determinada, permitiendo detectar posibles intentos de abuso del sistema.

WebConfig: configuración adicional de componentes web que complementa la configuración de seguridad, incluyendo configuraciones de CORS adicionales, manejo de recursos estáticos y configuraciones de filtros web.

Flujo de autenticación

El proceso de autenticación sigue un flujo controlado donde las credenciales son validadas por `AuthenticationService`, los tokens son generados por `JwtService`, y las peticiones subsecuentes son filtradas por `JwtAuthenticationFilter` para verificar la validez de los tokens y establecer el contexto de seguridad apropiado.

El flujo de recuperación de contraseña opera de forma independiente: el trabajador solicita el restablecimiento mediante su email, `PasswordResetTokenService` genera un token de un solo uso con expiración de 30 minutos, se envía al trabajador por email mediante `EmailService`, y al usarlo se marca como utilizado e invalida automáticamente para usos posteriores.

5.4.2.6 DTOs y transformación de datos

La aplicación implementa el patrón Data Transfer Object (DTO) para separar la representación interna de los datos (entidades JPA) de la representación externa utilizada en la comunicación con clientes.

CitaDTO: versión simplificada de la entidad Cita que incluye solo los campos necesarios para la mayoría de las operaciones, evitando la exposición de relaciones complejas.

CitaCompletaDTO: versión extendida que incluye información detallada del presupuesto y cliente, utilizada para operaciones que requieren una vista completa del servicio.

CitaModificacionDTO: DTO especializado para operaciones de modificación que incluye validaciones y campos específicos para cambios de fecha y hora.

CitaPresupuestoDTO: modelo de transferencia que combina información de cita con datos específicos del presupuesto, facilitando la gestión integrada de servicios y costes.

PreciosIndividualesDTO: DTO que encapsula el desglose detallado de precios por componentes, permitiendo mostrar al cliente la composición específica del coste total del servicio.

TrabajadorDTO: contiene información básica del trabajador sin exponer datos sensibles como contraseñas o información financiera, utilizado en listados y asignaciones de personal.

5.4.3. Generación y envío de facturas en formato PDF

El sistema incorpora un módulo automatizado de generación documental encargado de crear facturas en formato PDF a partir de la información gestionada dentro del sistema de citas. Para ello se utiliza la librería Apache PDFBox, que permite la construcción dinámica de documentos mediante programación, garantizando flexibilidad en el diseño y control total sobre la estructura visual del archivo generado.

El proceso comienza con la creación de un documento en formato A4 y la inicialización de un **PDPageContentStream**, componente responsable de permitir la escritura de texto, formas gráficas y elementos de maquetación directamente sobre la página del documento. Durante esta fase, los datos necesarios para la facturación se obtienen de forma dinámica desde la base de datos, incluyendo el precio base del servicio, recuperado mediante el repositorio de precios configurado en el sistema. Este mecanismo asegura la coherencia entre la configuración económica del sistema y los documentos emitidos.

La factura generada sigue la identidad visual corporativa del estudio **INK&CO Tattoo Studio**, incorporando una cabecera con fondo oscuro junto al logotipo del estudio cargado desde los recursos internos de la aplicación mediante el *class loader*. En esta misma sección se muestran el identificador de la factura y la fecha de emisión, organizados de forma alineada para mantener una presentación profesional y homogénea.

Posteriormente se incluyen los datos fiscales del estudio y del cliente, permitiendo que el documento cumpla con los requisitos administrativos habituales. Para mejorar la organización visual del contenido, el diseño emplea separadores gráficos y una distribución estructurada de la información.

El núcleo del documento está formado por una tabla de servicios generada dinámicamente, donde se reflejan tanto el precio base como los distintos parámetros configurables asociados a la cita —tipo de tatuaje, zona corporal, tamaño, nivel de detalle, coloración y estilo artístico— mostrando cada concepto con su correspondiente importe. La tabla aplica técnicas de mejora de legibilidad mediante cabeceras diferenciadas y filas alternadas.

En la sección final se realizan los cálculos económicos correspondientes, mostrando la base imponible, el impuesto sobre el valor añadido (IVA) aplicado y el importe total de la factura, resaltado visualmente mediante los colores corporativos definidos para la aplicación. La composición gráfica mantiene una coherencia estética basada en la identidad visual del sistema.

Respecto a la gestión tipográfica, el sistema utiliza la nueva API introducida en **PDFBox 3.x**, basada en la instanciación explícita de fuentes estándar mediante *PDType1Font* y *Standard14Fonts*, sustituyendo el modelo utilizado en versiones anteriores y garantizando compatibilidad futura con la librería.

Una vez finalizada la generación del documento, el PDF se almacena automáticamente en el servidor dentro de un directorio destinado a la gestión documental de facturas. Dicho directorio, denominado facturas/, se ubica en la raíz del proyecto y se crea de forma automática en el primer uso, sin requerir intervención manual. La ruta de almacenamiento se encuentra externalizada en el fichero `application.properties` mediante la propiedad `tatusys.facturas.ruta`, lo que permite modificarla sin necesidad de recompilar la aplicación. Cada archivo generado sigue la nomenclatura `factura_NombreApellido_ID.pdf`, facilitando su identificación y organización. Este almacenamiento local permite mantener una copia persistente de cada documento generado sin incrementar la carga estructural de la base de datos, facilitando además la recuperación o reenvío posterior en caso necesario.

Tras el almacenamiento, el sistema inicia el proceso de envío de la factura al cliente mediante el servicio de correo electrónico integrado en la aplicación. El envío se realiza utilizando **JavaMailSender** a través del protocolo SMTP de Gmail, adjuntando el documento directamente desde memoria mediante *ByteArrayResource*, evitando operaciones adicionales de lectura desde disco durante el proceso de envío y reduciendo el tiempo necesario para completar la operación.

En lo referente a la configuración del servidor de correo, durante la fase de desarrollo se utilizó Mailtrap como entorno de pruebas para la validación del envío de emails sin riesgo de impactar a destinatarios reales. Una vez verificado el correcto funcionamiento del módulo de notificaciones, se realizó la migración al servidor SMTP de Gmail, habilitando el envío de correos electrónicos reales desde una cuenta corporativa dedicada a la aplicación. Para

garantizar la seguridad de la integración, se configuró la verificación en dos pasos en la cuenta de Gmail y se generó una contraseña de aplicación específica para TatuSys, evitando así el uso de la contraseña principal de la cuenta y siguiendo las recomendaciones de seguridad de Google para accesos SMTP externos. La configuración resultante utiliza el host smtp.gmail.com en el puerto 587 con autenticación STARTTLS, quedando las credenciales externalizadas en el fichero application.properties para facilitar su gestión y mantenimiento.

Con el objetivo de optimizar el rendimiento del sistema, el envío de correos electrónicos se ejecuta de forma asíncrona mediante la anotación **@Async**. Esta configuración delega la tarea a un **ThreadPoolTaskExecutor** específico definido en la configuración de la aplicación, encargado de gestionar un conjunto controlado de hilos dedicados exclusivamente al procesamiento de notificaciones por correo electrónico.

Dicho ejecutor asíncrono mantiene varios hilos activos de forma permanente y permite el procesamiento simultáneo de múltiples envíos, incorporando además un sistema de cola interna para gestionar picos de solicitudes sin afectar al rendimiento general del servidor. Gracias a este mecanismo, las operaciones de comunicación quedan desacopladas del flujo principal de ejecución del backend.

Adicionalmente, se implementó una optimización del canal SMTP mediante la activación del **pooling de conexiones**, permitiendo reutilizar conexiones previamente establecidas con el servidor de correo en lugar de crear una nueva conexión para cada envío. Esta estrategia reduce significativamente la latencia asociada a la autenticación SMTP y mejora el rendimiento en escenarios con múltiples envíos consecutivos de facturas o notificaciones.

Como resultado, la aplicación puede devolver la respuesta HTTP al usuario de forma inmediata mientras el envío del correo se realiza en segundo plano. Esta arquitectura evita bloqueos en el hilo principal del servidor, mejora la capacidad de respuesta del sistema y permite escalar correctamente ante múltiples solicitudes simultáneas de generación y envío documental.

En conjunto, esta solución desacopla la generación documental del sistema de comunicación, optimiza el uso de recursos del servidor y aproxima el funcionamiento de la aplicación a modelos arquitectónicos empleados en soluciones empresariales reales de gestión y facturación digital.

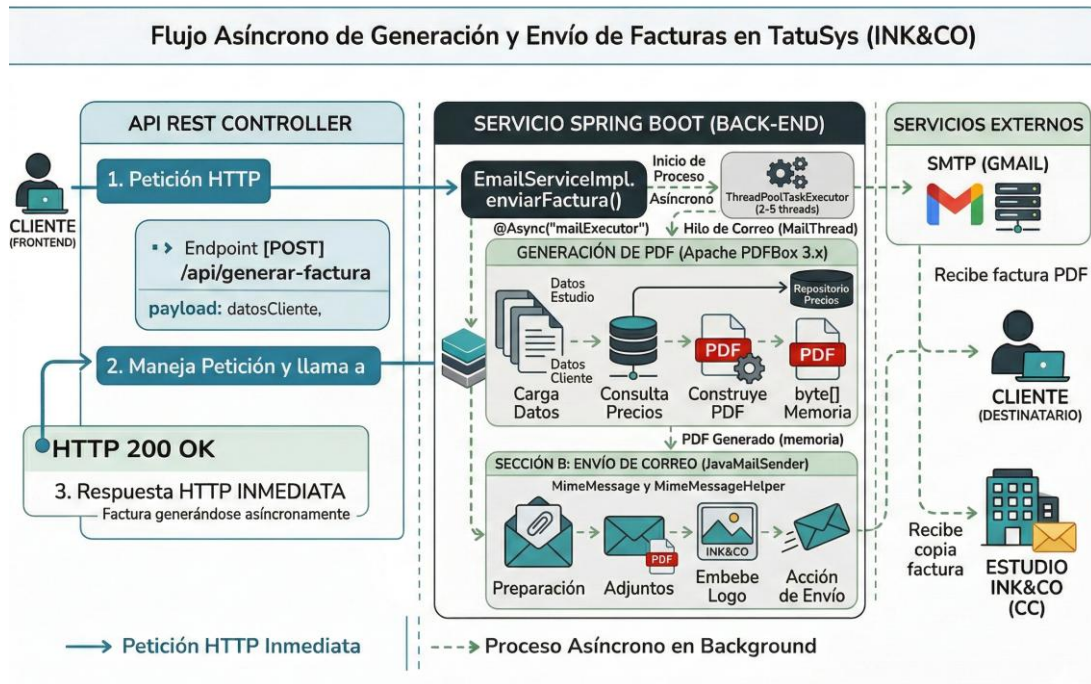


Figura 23. Imagen del flujo asincrónico del software.

La optimización mediante reutilización de conexiones SMTP no modifica el flujo arquitectónico representado en el diagrama, sino que mejora el rendimiento interno del proceso de envío manteniendo intacta la ejecución asincrónica del sistema.

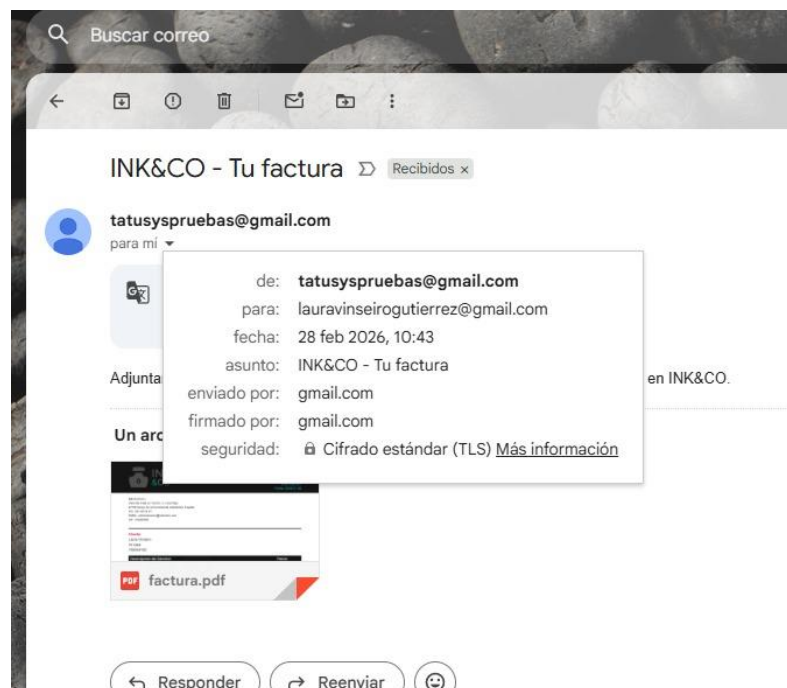


Figura 24. Imagen del buzón de envío del estudio.



INK

&CO

FACTURA

N de Cita: #31

Fecha: 2026-01-29

Calle del Pez, 14, Local Bajo

28004 Madrid, España

TEL: 621 89 76 27

EMAIL: administracion@inkandco.com

CIF: 12345678Z

Cliente:

laura laura

mi casa

78220410D

Descripción del Servicio	Precio
PRECIO BASE	50,00 EUR
ELIMINACION	20,00 EUR
CERVICAL	20,00 EUR
MEDIANO	40,00 EUR
MEDIO	25,00 EUR
NEGRO	0,00 EUR
LETTERING	0,00 EUR

Gracias por confiar en nuestro estudio de tatuajes.

Base Imponible:

Cuota IVA 21%:

TOTAL:

155,00 EUR

32,55 EUR

187,55 EUR

Figura 25. Aspecto del pdf de la factura.

5.5. Desarrollo del frontend (dashboard del tatuador)

5.5.1. Introducción al interfaz del tatuador

Para la parte del tatuador, nuestro cliente principal, hay que tener en cuenta lo siguiente: el profesional necesita recibir y organizar las citas con sus clientes, por lo que requiere contar con funciones de agenda y envío de presupuestos. Para ello la mejor opción es hacer un diseño tipo dashboard con una **Arquitectura Standalone** y **Lazy Loading** para una carga instantánea.

Como el flujo de las citas se basa en el estado del presupuesto (PENDIENTE, GENERADO, ACEPTADO, FINALIZADO y RECHAZADO) y en el estado del servicio (PENDIENTE, CONFIRMADO y FINALIZADO), nuestra aplicación debe emplear esos estados para saber que citas mostrar y dónde hacerlo.

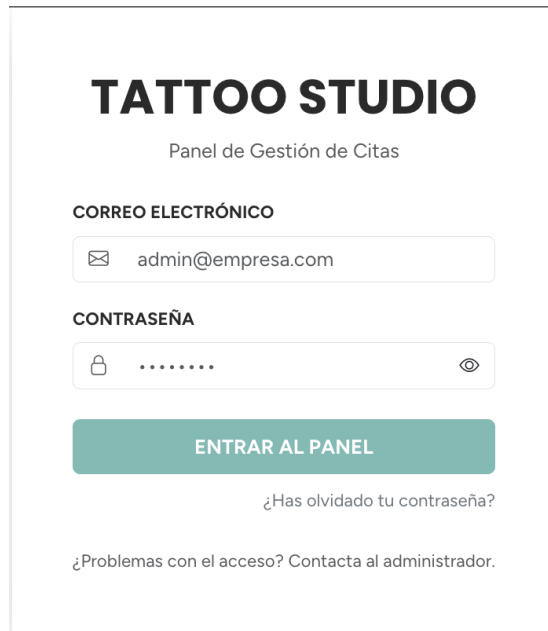
Con el fin de optimizar el desarrollo del dashboard, hemos usado componentes genéricos, ya que muchas páginas tienen la misma estructura con pequeños cambios. Por lo tanto, tenemos **Appointment-Modify** (para editar las citas) y **Appointment-Detail** (para ver los detalles de la cita). Así reutilizamos el código y tenemos una estructura más limpia. Además de estos componentes variables hay otros dos elementos que siempre están a la vista en el dashboard: el **Sidebar** (de escritorio o móvil) y el **Breadcrumb** o migas de pan (nos muestra la ruta de la página en la que estamos).

Para poder mostrar siempre estos dos elementos hemos usado un componente padre que hace uso de **Router Outlet**. El uso de esta directiva es crucial, ya que es un ancla que nos permite mostrar las diferentes páginas de forma dinámica. Este mismo concepto está aplicado a aquellas páginas que tienen subpáginas, como solicitudes, home, etc.

Otro punto importante son los roles de los usuarios. Tenemos dos: **administrador** y **trabajador**. El primero tendrá acceso a todas las páginas y subpáginas (home, solicitudes, calendario, trabajadores y configuración), puesto que se trata del propietario y tiene la autoridad para gestionar tanto la plantilla como los precios del establecimiento. Por su parte, el tatuador tendrá acceso solo al home, solicitudes y calendario.

Con el objetivo de lograr que cada usuario acceda a sus secciones correspondientes, hemos hecho uso de un sistema de inicio de sesión de cuentas. Así, además de filtrar las páginas según el rol, también se filtran las citas asignadas a la persona que ha iniciado sesión. Aquí es importante el concepto **Authorization Token**. Con este token almacenamos la información del

usuario y de este modo sabemos si es administrador o tatuador. Para iniciar sesión, el usuario debe poner su correo electrónico y su contraseña. De esta manera y con el uso de Guards de doble nivel (Auth/Admin) protegemos de los trabajadores autorizados aquellas páginas que solo son para los administradores.



The screenshot shows a login form for 'TATTOO STUDIO'. The title 'TATTOO STUDIO' is in large, bold, black letters. Below it, the subtitle 'Panel de Gestión de Citas' is in a smaller font. The form has two main sections: 'CORREO ELECTRÓNICO' and 'CONTRASEÑA'. The 'CORREO ELECTRÓNICO' section has a text input field with an email icon and the text 'admin@empresa.com'. The 'CONTRASEÑA' section has a text input field with a lock icon, a password mask '.....', and an eye icon to toggle visibility. Below these fields is a teal button labeled 'ENTRAR AL PANEL'. At the bottom, there are two links: '¿Has olvidado tu contraseña?' and '¿Problemas con el acceso? Contacta al administrador.'

Figura 26. Captura del formulario de inicio de sesión del dashboard.

También, tenemos una sección para aquellos usuarios que han olvidado su contraseña, y pueden acceder mediante el enlace “¿Has olvidado tu contraseña?”.



The screenshot shows a password recovery form for 'TATTOO STUDIO'. The title 'TATTOO STUDIO' is in large, bold, black letters. Below it, the subtitle 'Recuperar Acceso' is in a smaller font. The form has a text input field for the email address with the text 'Introduce tu email de trabajador para recibir un enlace de recuperación.' above it. The input field contains the email 'trabajador@tatusys.com'. Below the input field is a teal button labeled 'ENVIAR ENLACE'.

Figura 27. Recuadro de recuperación de contraseña.

Este enlace nos lleva a una página donde se le pide al usuario que ponga su email corporativo de TatuSys y allí se le enviará un enlace con su token a una página para restablecer su contraseña de la base de datos.

TATTOO STUDIO

Establecer Nueva Contraseña

NUEVA CONTRASEÑA

🔑 Mínimo 6 caracteres

CONFIRMAR CONTRASEÑA

🛡️ Repite tu contraseña

CAMBIAR CONTRASEÑA

Figura 28. Recuadro de nueva contraseña.

En cuanto al diseño del dashboard, nos centramos en uno minimalista e intuitivo para el usuario que ayuda a simplificar la curva de aprendizaje. Puesto que se trata de un dashboard con diferentes apartados, es necesario tener un menú lateral (en escritorio) y superior (en móvil) para poder navegar cómodamente entre las diferentes páginas.

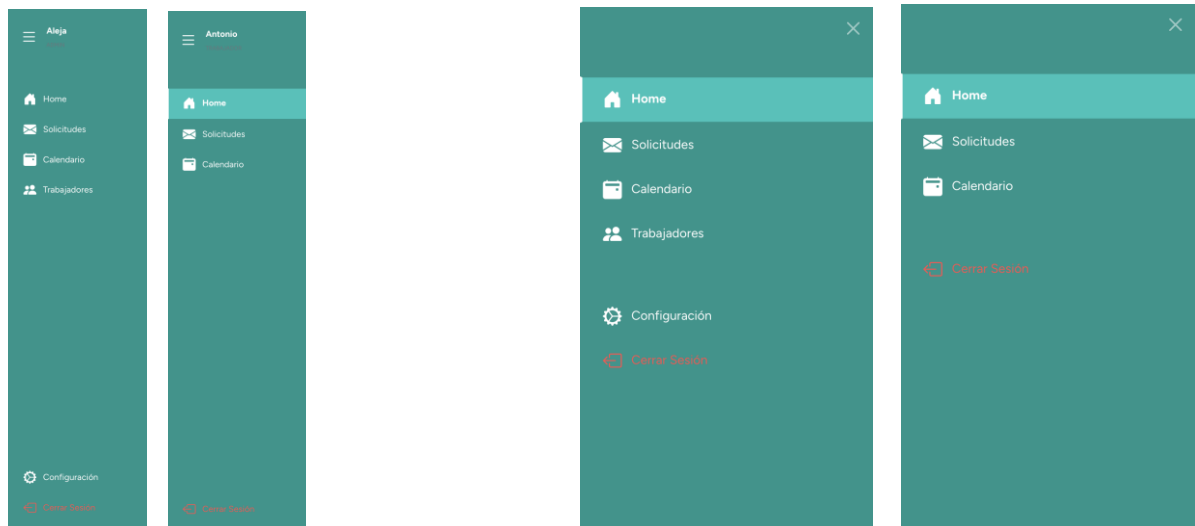


Figura 29. Visualización del menú de navegación en escritorio (izq.) y móvil (der.).

A continuación, se explica el funcionamiento de cada una de las secciones del panel de administración, siguiendo el flujo natural de las citas de principio a fin. Más adelante, se

explica también el funcionamiento de aquellas páginas que no tienen una relación tan directa con el flujo de las citas, pero que también resultan relevantes.

5.5.2. Página de Solicitudes

Esta página es el apartado dónde llegan las nuevas solicitudes de cita hechas por los clientes en la página web pública. Dentro de esta página tenemos los siguientes apartados, acompañados de botones de navegación superiores.

Apartado de solicitudes pendientes: aquí es donde llegan las nuevas solicitudes, con el estado de presupuesto y el de servicio pendientes. Las peticiones se visualizan mediante una tabla que muestra los detalles principales para poder identificarlas haciendo uso de **Pipes personalizados** (id, nombre del cliente, fecha/hora y tipo de servicio). Y para que se muestren en esta tabla se usa el método de mostrar todos, pero siguiendo las condiciones de los estados del presupuesto y el servicio. El estilo de esta página se repite en otras, por eso hemos usado **Encapsulation None** para poder reciclar código.

#	Nombre	Fecha y Hora	Tipo	
5	Marta	2026-03-23 (11:00)	TATUAJE	Revisar
6	Marta	2026-03-23 (13:30)	TATUAJE	Revisar

Figura 30. Captura del apartado “Solicitudes Pendientes” del dashboard.

Esta página, además de mostrar las solicitudes, sirve tanto para revisar los datos de estas y sus presupuestos como para enviar estos últimos al cliente. Esto es porque la solicitud que envía el cliente llega con un presupuesto que se genera automáticamente, pero es necesario que el

tatuador lo revise, ya que a veces los diferentes apartados del servicio que selecciona el cliente podrían no coincidir del todo con la referencia visual adjunta. De esta forma, con el botón “Revisar” accedemos a una página donde se muestra de forma detallada la información de la cita.

Revisar presupuesto: en este recuadro encontramos tres secciones: datos del cliente (solo visualización), servicio (visual y editable), y presupuesto (visual y editable).

Aquí, el tatuador puede modificar los datos de la posible cita con unos botones tipo *select* que permiten modificar dinámicamente el precio de los elementos y ver cómo se actualiza el presupuesto. Para conseguir esto hicimos uso de **Forms Module**, que es un **Módulo de Funcionalidad (Feature Module)** que forma parte de la biblioteca oficial `@angular/forms`. Con esto y una función que actualiza el precio en memoria conseguimos que se visualice el cambio de precio total al cambiar los valores de cada elemento dotado de un botón *select*, lo cual resulta más amigable para la experiencia del usuario. También es necesario el uso de `ForkJoin`, un operador `RxJS` que permite ejecutar múltiples peticiones HTTP en paralelo y esperar a que todas terminen para devolver los resultados juntos en un solo bloque.

En el apartado de presupuesto, el tatuador puede incluir comentarios para el cliente y aceptar o rechazar el presupuesto. En caso de que lo acepte (independientemente de si ha hecho modificaciones de algún dato o no), los datos se almacenan, el estado del presupuesto pasa a **generado** y el estado de la cita permanece en **pendiente**. La otra opción es que el tatuador lo rechace por alguna razón, lo cual elimina la solicitud y la posibilidad de llegar a realizar la cita. En este caso, el estado del presupuesto pasaría a rechazado.

Llegados a este punto, la cita pasaría a la sección de solicitudes revisadas y se envía automáticamente un correo electrónico al cliente con el presupuesto, indicaciones útiles y un formulario para pagar la fianza. Más adelante entraremos más a fondo en esto.

Solicitudes > Pendientes > Revisar

Confirmar Servicio - #6

Datos del cliente

Nombre:
Marta

Apellidos:
Munera

Teléfono:
6786786778

Email:
santimuner9@gmail.com

DNI/NIF:

Servicio

Precio Base:

Tipo: Tatuaje

Zona: Hombro

Tamaño: Grande

Detalle: Medio

Coloración: Color

Estilo: Realismo

Duración estimada: 4h 30 min

COMENTARIOS:

Lo quiero muy realista

Imagen 152d96ab8-1aa5-4857-8c83-23f623ec210d.png

Presupuesto

COMENTARIOS:

Precio Base: €155.00

IVA (21%): €32.55

TOTAL ESTIMADO: €187.55

Estado: PENDIENTE

Fecha y Hora: 23/03/2026 13:30

Trabajador: Antonio

ACTUALIZAR PRESUPUESTO

RECHAZAR PRESUPUESTO

Figura 31. Vista del recuadro de edición/revisión de los detalles de una solicitud.

Sección de solicitudes revisadas: esta página es parecida a la anterior, ya que muestra una tabla con las citas, pero en este caso, con el presupuesto ya enviado al cliente. Aquí, las solicitudes tienen un tiempo de vida de 48 horas, a la espera de que el cliente pague la fianza y se proceda con la confirmación. Por lo tanto, todo lo que encontremos en esta página será visual e informativo, sin posibilidad de edición.

Solicitudes > Revisadas

Pendientes Revisadas

#	Nombre	Fecha y Hora	Tipo	
1	Marta	2026-03-16 (11:00)	TATUAJE	Ver
7	Pedro	2026-03-24 (11:00)	COVER	Ver
8	Marcos	2026-03-24 (16:00)	ELIMINACION	Ver

Anterior

1

Siguiente

Figura 32. Vista de las solicitudes revisadas en el dashboard.

Además de la tabla de citas revisadas hay un botón “Ver” para mostrar la información detallada de estas.

Solicitudes > Revisadas > Detalle

Reserva - #8

Datos del cliente

Nombre:
Marcos

Apellidos:
Gutierrez

Teléfono:
589695956

Email:
marcos@gmail.com

DNI/NIF:

Servicio

Precio Base:

Tipo: ELIMINACION

Zona: ANTEBRAZO

Tamaño: PEQUEÑO

Detalle: SENCILLO

Coloración: NEGRO

Estilo: LETERING

Duración del Servicio:
1h 30min

COMENTARIOS:

Sin comentarios adicionales.

Presupuesto

COMENTARIOS:

Sin comentarios adicionales.

Estado: **GENERADO** Fecha y Hora: 24/03/2026 16:00 Trabajador:
Aleja

Precio Base:
€130.00
IVA (21%):
€27.30
Precio Final:
€157.30

Figura 33. Vista del recuadro de información detallada de una solicitud revisada.

Por otro lado, tenemos el correo electrónico que se le envía al cliente una vez el tatuador haya revisado el presupuesto. Para poder simular todo este proceso hicimos uso de **Mailtrap** en relación con la API, para que se pudieran atrapar todos los correos que se envíen sin importar la dirección de correo electrónico, y así evitar el problema de hacer pruebas con direcciones que pudieran estar en uso. Al final del desarrollo implementamos el envío de estos correos a las cuentas de correo electrónico que indica el cliente en la solicitud del servicio sin hacer uso de Mailtrap.

Este correo se envía para que el cliente pueda ver el importe total del servicio solicitado. El formato de este correo está maquetado de modo que coincida con el diseño visual de todo el proyecto, dotando de coherencia a la imagen de marca del estudio INK&CO. Para conseguir esto, hicimos uso de **HTML Corporativo** desde la API (EmailService). Para el envío del correo utilizamos **Java Mail Sender**, una **interfaz** de Spring Framework (específicamente de la biblioteca spring-context-support) que extiende las capacidades de la API estándar de JavaMail para simplificar el envío de correos electrónicos.

En este correo se le muestra al cliente el localizador de su cita, el importe total y la fianza que debe pagar para aceptar y confirmar su cita. Además, se le notifica que tiene 48 horas para hacerlo. También hay un botón con un enlace a la página para que el cliente pueda pagar su fianza.

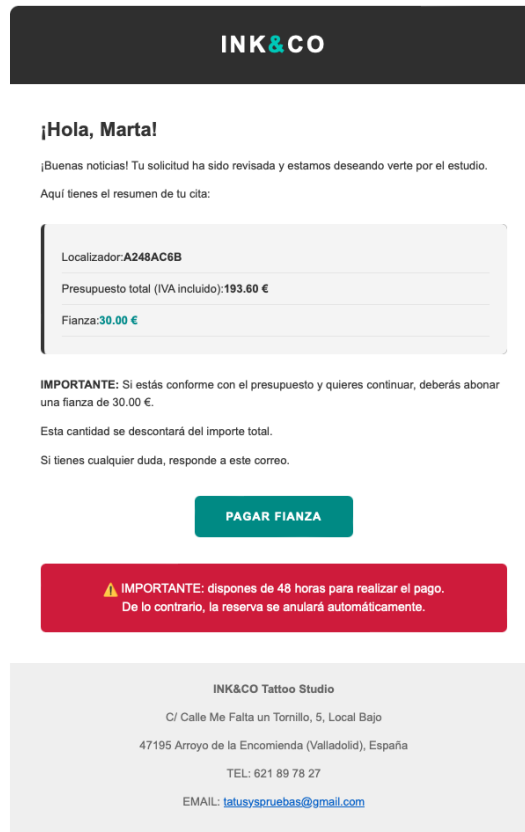


Figura 34. Imagen del cuerpo de un correo electrónico en la fase de pruebas.

Página de pago de la fianza: aquí, además de mostrar los datos de la cita, hay un formulario para que el cliente ponga los datos de la tarjeta en dónde se cobrará la fianza. Ahora mismo esta página está en modo simulación. Eso quiere decir que no se le cobra nada a la tarjeta que se ponga en el formulario. Solo se verificará que los datos se inserten de forma correcta y sean coherentes (fecha de tarjeta que no puede ser de un año anterior, CVC con un número de cifras no inferior a 3, etc.). Cuando el cliente completa este “pago” dentro del plazo requerido de 48 horas, la cita queda confirmada, lo cual actualiza el estado del presupuesto a aceptado y el del servicio a confirmado. También puede rechazar la propuesta de presupuesto si no desea continuar con el proceso.

La imagen muestra dos paneles de interfaz de usuario. El panel izquierdo, titulado 'DETALLES DE TU CITA', contiene un formulario con los siguientes campos: 'Localizador:' con el valor '78CC1794', 'Fecha:' con '23/03/2026', 'Hora:' con '13:30:00', 'Importe total:' con '187.55 €', y 'Fianza:' con '30 €'. Debajo de estos campos, hay un texto '(A descontar del importe total)'. En la parte inferior del panel, hay dos mensajes: uno en un recuadro azul que dice 'El resto del importe se abonará en el estudio el día de la cita.' y otro en un recuadro rojo que dice 'Tienes 48 horas para realizar el pago. Al realizar el pago, aceptas el presupuesto y la fecha. Si no realizas el pago en 48 horas, la solicitud quedará anulada.' El panel derecho, titulado 'DATOS DE PAGO', tiene un encabezado 'MODO SIMULACIÓN TFG'. Contiene campos para 'Titular de la tarjeta' (Nombre completo del titular), 'Número de tarjeta' (0000 0000 0000 0000), 'Caducidad' (MM/AA) y 'CVC' (123). En la parte inferior, hay dos botones: 'REALIZAR PAGO (30 €)' y 'RECHAZAR PRE-RESERVA'.

Figura 35. Captura de los recuadros de información y datos de pago disponibles al usar el enlace de pago.

5.5.3. Home (página de citas confirmadas)

En esta página tenemos como contenido principal la tabla de las citas confirmadas, es decir, aquellas para las que se ha completado el flujo explicado anteriormente. Hacemos uso de un método parecido a las anteriores secciones, pero en este caso las citas que se muestran tienen el estado del presupuesto aceptado y el de servicio confirmado.

Además, tenemos dos secciones que informan al tatuador de la cantidad de nuevas citas que tiene confirmadas y también de la cantidad de solicitudes nuevas que tiene por revisar. Esta última sirve como enlace a la página de solicitudes.

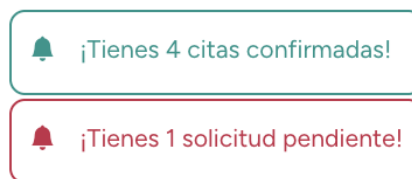


Figura 36. Recuadros de notificación de novedades en el Home del dashboard.

En esta sección de la aplicación mostramos la agenda del tatuador. Dispone de unos botones superiores que permiten mostrar las citas en un día o en una semana y haciendo uso de la **inyección de estado** recuerda en qué fecha estábamos. Además, hay otros dos botones superiores que sirven para avanzar o retroceder cronológicamente, independientemente de

la vista en la que esté. En cada fila correspondiente a una cita, el usuario dispone de los botones “Finalizar” y “Ver”, que permiten acceder a subsecciones de visualización de las citas. También, para que la página sea responsive hemos hecho uso de **HostListener**, un decorador que detecta cuándo el usuario cambia el tamaño de la ventana.

The screenshot shows a web application interface. On the left is a teal sidebar with icons for home, mail, calendar, users, settings, and a logout button. The main content area has a header with a 'Home' link, a date range '12 mar - 18 mar', and buttons for 'Día', 'Semana', and navigation arrows. Below the header is a table with columns: '#', 'Nombre', 'Fecha y Hora', 'Tipo', and two action buttons 'Finalizar' and 'Ver'. The table contains one row with data: '# 2', 'Nombre Marta', 'Fecha y Hora 2026-03-16 (12:00)', 'Tipo COVER', and buttons 'Finalizar' and 'Ver'. At the bottom, there are two notification boxes: a green one saying '¡Tienes 1 cita confirmada!' and a red one saying '¡Tienes 1 solicitud pendiente!'. On the right, there are navigation buttons 'Anterior', '1', and 'Siguiente'.

#	Nombre	Fecha y Hora	Tipo		
2	Marta	2026-03-16 (12:00)	COVER	Finalizar	Ver

Figura 37. Vista de la tabla de próximas citas y las notificaciones en la zona inferior.

Ver detalles de la cita confirmada: esta subpágina es igual que la página de detalles de la solicitud explicada anteriormente, muestra los datos completos de la cita.

Finalizar cita: aquí es donde se confirma que el servicio se ha llevado a cabo. Es igual que la página de revisar presupuesto, pero con unas pequeñas diferencias. En esta página, el tatuador puede modificar el presupuesto en caso de que haya surgido algún imprevisto. Además, tiene otro botón para finalizar la cita.

En el proceso de finalizar la cita hay varias cosas a tener en cuenta. Cuando la finalizamos, el estado del presupuesto y el servicio pasan a finalizado. Así, ya no se mostrarán en ninguna tabla del dashboard. Por otro lado, tenemos la facturación de las citas. Para esto tenemos un estado de factura dentro de la entidad servicios: **pendiente, no requiere y enviada**.

Cuando el cliente solicita su cita, dentro del formulario tiene un botón para indicar si quiere factura o no. Si no lo selecciona, la solicitud se genera con un estado de factura “no requiere”. Si lo llega a seleccionar, el estado de la factura sería “pendiente”.

Una vez el tatuador finalice la cita con el botón mencionado anteriormente, para generar una factura se le pedirán los datos fiscales del cliente y el estado de factura pasará a “enviada”.

Home > Confirmar Presupuesto

Confirmar Servicio - #2

Datos del cliente

Nombre:
Marta

Apellidos:
Munera

Teléfono:
6786786778

Email:
santimuner9@gmail.com

DNI/NIF:

Servicio

Precio Base:

Tipo:
Cover

Zona:
Rodilla

Tamaño:
Grande

Detalle:
Medio

Coloración:
Color

Estilo:
Blackandgrey

Duración estimada:
4h 30 min

COMENTARIOS:
Sin comentarios adicionales.

Presupuesto

COMENTARIOS:

Estado: **ACEPTADO** Fecha y Hora: 16/03/2026 12:00 Trabajador: Aleja

Precio Base: €160.00
IVA (21%): €33.60
TOTAL ESTIMADO: €193.60

ACTUALIZAR PRESUPUESTO

FINALIZAR CITA

Figura 38. Vista del apartado para finalizar una cita.

Cuando finalizamos la cita tenemos dos opciones:

- Si el cliente había marcado que quería factura, nos aparece un mensaje de que el cliente quería factura. Por lo tanto, nos obliga a rellenar el formulario con los datos del DNI y Dirección Fiscal.

Home > Confirmar Presupuesto > Generar Factura

¿Quieres crear una Factura?

La factura se le enviará al cliente por Email.

⚠ Esta cita requiere factura obligatoriamente.

DNI / NIE

Ej: 12345678X

Dirección Fiscal

Calle, Número, CP...

Generar

Figura 39. Página de creación de factura por petición del cliente.

- Si el cliente no quería factura, le aparece una página al tatuador donde se le pregunta si el cliente va a querer factura, por si ha cambiado de opinión. En caso de que quiera, debe rellenar el DNI del cliente y su Dirección Fiscal. Luego, al generarla, se le envía la factura al email del cliente. Si no quiere, solo debe usar el botón de "No generar".

Home > Confirmar Presupuesto > Generar Factura

¿Quieres crear una Factura?

La factura se le enviará al cliente por Email.

DNI / NIE

Ej: 12345678X

Dirección Fiscal

Calle, Número, CP...

No Generar Generar

Figura 40. Página de inserción de datos para la factura.

Gracias a **Angular Signals** conseguimos mostrar una cosa u otra dentro del mismo componente.

5.5.4. Calendario

Esta página muestra las citas confirmadas que tiene el tatuador cada mes, mediante una cuadrícula de un mes completo con sus celdas correspondientes. Al hacer clic en una cita, se abre el recuadro que muestra los detalles a modo informativo de referencia. También tenemos unos botones superiores que permiten avanzar y retroceder en el tiempo. Para conseguir esto hemos hecho uso de **Full Calendar Component** (el objeto visual que ponemos en el HTML) y **Full Calendar Module** (es el puente técnico, sirve para organizar las piezas de funcionalidad y decirle al compilador qué herramientas tiene disponibles).

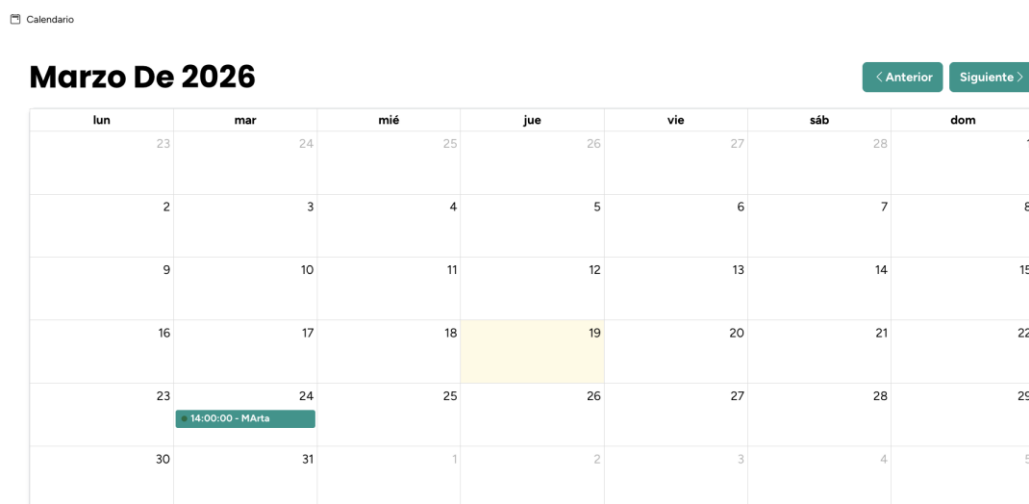


Figura 41. Vista del calendario del dashboard.

5.5.5. Trabajadores (configuración del personal)

Como hemos mencionado anteriormente, a esta página solo tendrán acceso los usuarios con rol de administrador. La función principal de esta página es proporcionar un **CRUD (create, read, update and delete)** para la gestión de empleados.

De este modo, se muestra la lista de trabajadores con los datos más importantes para identificarlos. El usuario dispone de un botón "Editar", un botón "Eliminar" y el botón "Añadir Staff" para dar de alta a nuevos empleados.



Trabajadores

Gestión del Equipo (Staff)

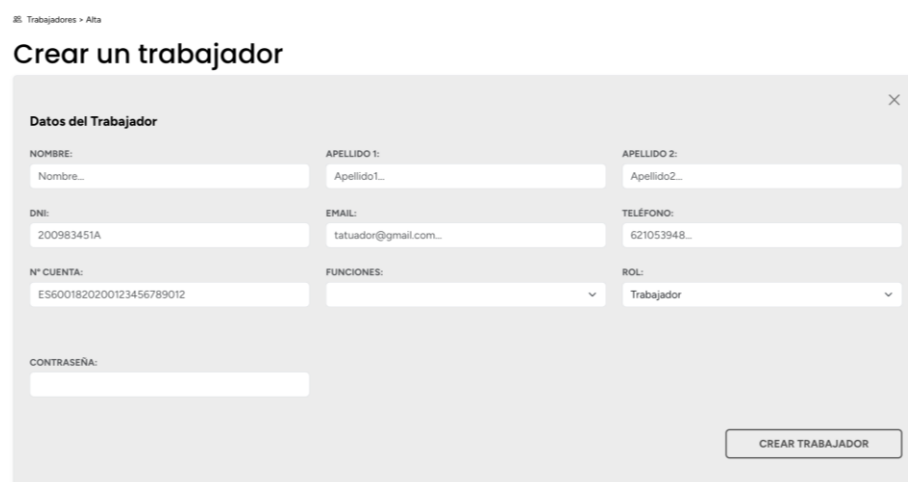
[Añadir Staff](#)

ID	Nombre	Email	Tel.		
3	Antonio Morales	duplicado2@test.com	612345678	Editar	Eliminar
7	Santiago Múnera	santiago@gmail.com	621051292	Editar	Eliminar

[Anterior](#) [1](#) [Siguiente](#)

Figura 42. Vista de la sección de gestión de los trabajadores.

Crear un trabajador: hay un formulario principal para insertar los datos del tatuador: nombre, apellidos (el segundo apellido no es obligatorio), DNI, email, teléfono, número de cuenta bancaria, funciones (es decir, el tipo de trabajo que desempeñan), rol (en este caso solo trabajador) y la contraseña para la cuenta. La contraseña que insertemos se encriptará una vez que demos de alta al trabajador. Con el botón "Crear trabajador" confirmamos el alta y guardamos los datos en la base de datos.



Trabajadores > Alta

Crear un trabajador

Datos del Trabajador

NOMBRE:

APELLIDO 1:

APELLIDO 2:

DNI:

EMAIL:

TELÉFONO:

N° CUENTA:

FUNCIONES:

ROL:

CONTRASEÑA:

[CREAR TRABAJADOR](#)

Figura 43. Vista del recuadro de alta de nuevos trabajadores.

Editar trabajador: en esta subpágina encontramos un formulario parecido al de crear un trabajador. En este caso, se muestran los campos rellenos con los datos del trabajador, y está la opción de modificarlos. Hay dos botones inferiores: uno para revertir los cambios (si se ha cambiado algo con este botón restablecemos los datos con los que hay en la base de datos) y actualizar cambios (confirma que queremos guardar los nuevos datos en la base de datos y solo se activa si hemos cambiado algo).

Trabajadores > Editar

Editar Trabajador - # 3

Datos del Trabajador

NOMBRE:

Antonio

DNI:

99887766C

N° CUENTA:

ES1234567890123456789012

CONTRASEÑA:

APELLIDO 1:

Morales

EMAIL:

duplicado2@test.com

FUNCIONES:

Eliminación

APELLIDO 2:

TELÉFONO:

612345678

ROL:

Trabajador

REVERTIR CAMBIOS

SIN CAMBIOS

Figura 44. Vista del recuadro de modificación de datos de un trabajador.

Eliminar trabajador: cuando se pulsa el botón de eliminar aparece una página para confirmar la acción. Además, muestra el identificador y nombre del trabajador para que el usuario tenga claro a quien va a eliminar.

Trabajadores > Eliminar

Eliminar Trabajador



¿Estás seguro de que quieres eliminar a este trabajador?

ID TRABAJADOR:

#7

NOMBRE COMPLETO:

Santiago Múnera Preciado

Esta acción no se puede deshacer y el trabajador perderá el acceso al sistema.

CANCELAR

SÍ, ELIMINAR TRABAJADOR

Figura 45. Vista del recuadro de eliminación de un trabajador.

5.5.6. Configuración

A esta página solo tendrá acceso aquella cuenta que tenga el rol de administrador. Dentro de esta hay dos secciones: Administrador y Servicios. La primera es la vista por defecto de este apartado del dashboard.

Administrador: sirve para modificar los datos de la cuenta. Por lo tanto, hay un formulario idéntico en aspecto al de los trabajadores.

Ajustes > Administrador

Administrador Servicios

Editar Trabajador - # 1

Cambios detectados sin guardar

Datos del Trabajador

NOMBRE:	APELLIDO 1:	APELLIDO 2:
Aleja	Munera	Cardona
DNI:	EMAIL:	TELÉFONO:
20080892A	pedro.jimenez@tatusys.com	678678678
N° CUENTA:	FUNCIONES:	ROL:
ES2229292923922	Eliminación	Administrador
CONTRASEÑA:		

REVERTIR CAMBIOS ACTUALIZAR DATOS

Figura 46. Vista del recuadro de modificación de datos del administrador.

Servicios: esta página es crucial para el estudio de tatuajes, ya que influye en todas las citas. Aquí es donde el administrador pone los servicios que ofrece y sus precios. Por ejemplo: tipo (tatuaje) y precio (20 €), zona (brazo) y precio (30 €), etc. Aparte de estos servicios y precios específicos está el precio base (servicio base) que es el que se aplica a todas citas, es decir, el precio inicial. También se puede editar, pero está separado del resto para poder diferenciarlo por la importancia que tiene. Para conseguir mostrar todo esto y editarlo hacemos uso de **Forms Module**.

⊙ Ajustes > Servicios

Administrador Servicios

Configuración de Tarifas

Tarifa Mínima del Estudio
Precio base inicial para cualquier presupuesto.

0 € Guardar

COLORACION

NEGRO 10 € ✕ COLOR 25 € ✕ +

DETALLE

SENCILLO 0 € ✕ MEDIO 20 € ✕

DENSO 40 € ✕ +

ESTILO

ANIME 15 € ✕ BLACKANDGREY 0 € ✕

FINELINE 20 € ✕ LETERING 10 € ✕

JAPONES 25 € ✕ REALISMO 35 € ✕

TRADICIONAL 10 € ✕ +

Descartar cambios Confirmar y Guardar (0)

TAMANIO

Figura 47. Vista del recuadro de edición de los precios del establecimiento.

Como se puede ver en la imagen, hay diferentes elementos para poder insertar o eliminar servicios y precios. Siempre que se quiera meter un nuevo servicio, este debe contar con un precio, aunque sea 0 €, dado que están relacionados y son imprescindibles para el cálculo del presupuesto. Al igual que en páginas anteriores, se pueden descartar los cambios que se hayan hecho con el botón "Descartar cambios", haciendo uso del patrón **Dirty Checking Manual**, una manera de comparar manualmente el estado actual de un objeto con una foto (snapshot) de su estado original. Y para confirmarlos, hacemos uso del botón "Confirmar y Guardar", que solo se activará si hay cambios. De esta forma se guardarán los datos. Sin embargo, para guardar el precio base hay un botón aparte que únicamente guarda ese dato.

6. CONCLUSIONES

6.1. Conclusiones generales del proyecto

6.1.1. Conclusiones comunes

En términos generales, estamos satisfechos con los logros conseguidos: hemos sido capaces de crear una aplicación con el núcleo de funcionalidades que habíamos planteado y un funcionamiento satisfactorio para la gestión de citas y la creación de presupuestos, que eran los objetivos primordiales del proyecto.

Además, lo hemos logrado enfrentándonos a tecnologías con las que aún tenemos una experiencia limitada y supliendo la falta de conocimiento con ingenio y un enfoque práctico. Si bien se han producido algunos errores por nuestra inexperiencia, creemos que el resultado es muy digno dadas las circunstancias y que el aprendizaje ha sido provechoso.

Algunos aspectos del proyecto no se han podido implementar, o hemos tenido que descartarlos por razones de tiempo o capacidades técnicas, como es el caso del despliegue en un servidor (nuestros conocimientos aún son limitados), la creación de un editor de horarios para cada trabajador del estudio (puesto que carecíamos de tiempo para llevarlo a cabo por los retrasos que nos ocasionaron ciertos bugs en otras funcionalidades) o la implementación de una pasarela de pago real (demasiado compleja para nuestras capacidades actuales).

Tampoco se han realizado pruebas unitarias en el proyecto por una combinación de falta de tiempo y falta de base teórica, ya que es un ámbito en el que apenas hemos tenido formación, lo cual hemos suplido con pruebas manuales. Hemos decidido ceñirnos en todo momento a aquello que considerábamos que podíamos solventar con los medios a nuestro alcance y nuestra disponibilidad de tiempo real.

6.1.2. Conclusiones a título personal (Patricia)

En lo que respecta mi experiencia con el proyecto, creo que me ha enseñado la importancia de dedicar un plazo amplio a investigar qué se quiere hacer y cómo, y a crear un diseño sólido antes de plasmarlo en cualquier tipo de código, para minimizar todo lo posible problemas a medida que se avanza, por no haber contemplado algún aspecto. Nunca se puede prever todo,

pero profundizar bien en un proyecto antes de ponerlo en marcha ahorra muchos contratiempos.

Ha sido un reto desarrollar en Angular, una tecnología nueva para mí y que en la formación se ha visto de forma sucinta y veloz, como exige el calendario. Estoy satisfecha con haber ido comprendiendo cada vez mejor lo que iba haciendo, y creo que en futuros proyectos haría algunas cosas de forma diferente una vez adquirida la experiencia.

Ha sido un desafío ir solucionando problemas, errores inesperados y fallos que retrasaban el avance y que seguramente hoy día sabría solventar mejor. Uno de mis obstáculos ha sido mi propio equipo, cuya configuración de codificación de texto genera problemas de caracteres corruptos en diversos IDE y que, a pesar de las correcciones, siempre vuelve a generar algún problema que interfiere con el trabajo.

El proyecto también me ha servido para entender lo complejo que es ponerse en el lugar del usuario, es muy fácil obviar detalles que para otra persona son evidentes, y no todos navegamos ni pensamos de la misma forma al enfrentarnos a una interfaz. Es sin duda uno de los aspectos que me resulta más atractivos del desarrollo y lo que he aprendido me alienta a seguir mejorando lo que sé hacer y cómo planteo nuevos diseños de aquí en adelante.

6.1.3. Conclusiones a título personal (Santiago)

Al desarrollar este proyecto he logrado desarrollar una interfaz administrativa fluida y profesional, resolviendo desafíos complejos de maquetación para que el calendario y las tablas sean plenamente funcionales en cualquier dispositivo.

Implementar el control de accesos por roles y el flujo de recuperación de contraseñas me ha permitido entender la importancia de proteger la lógica de negocio y los datos sensibles de los trabajadores.

La integración de un backend con Hibernate y un frontend modular con Angular me ha enseñado a construir aplicaciones escalables, donde la navegación y el estado de la web son siempre coherentes.

Este proyecto ha sido un reto constante de resolución de problemas, desde depurar rutas hijas hasta sincronizar tokens de seguridad, dándome la confianza necesaria para afrontar desarrollos de software reales.

Por lo tanto, a pesar de los problemas técnico que hayan podido surgir me siento satisfecho con este proyecto porque he podido entender cómo funciona una aplicación completa. Desde la base de datos hasta la interfaz del usuario. Y eso es lo que quería con este grado superior, entender todo el camino de una aplicación web y saber hacerlo por mi cuenta.

6.1.4. Conclusiones a título personal (Laura)

El desarrollo de TatuSys ha supuesto para mí una experiencia especialmente valiosa, tanto a nivel técnico como personal. Mi rol principal fue la coordinación y desarrollo del backend, abarcando el diseño de la base de datos, la implementación de la API REST con Spring Boot y la integración con el resto del equipo.

Desde el punto de vista técnico, el proyecto me ha permitido consolidar conocimientos que hasta ahora había trabajado de forma más fragmentada: la gestión de autenticación y autorización, la generación de documentos, el manejo de notificaciones por correo o la organización de una arquitectura en capas, entre otros. Afrontar estas funcionalidades en un contexto real, con dependencias entre compañeros y plazos definidos, le ha dado a ese aprendizaje una dimensión que el estudio individual no puede replicar.

Una de las principales dificultades que encontramos como equipo fue la ausencia de una planificación inicial sólida. Al no tener experiencia previa en proyectos de esta envergadura, subestimamos la importancia de definir bien los requisitos, los flujos de trabajo y los tiempos desde el principio. Esto derivó en que durante el desarrollo surgieran imprevistos que había que resolver sobre la marcha, afectando a la coordinación y a los ritmos de entrega. Con la perspectiva que da haber terminado el proyecto, reconozco que establecer esa base desde el inicio habría hecho el proceso más ágil y menos costoso para todos.

En definitiva, TatuSys me ha enseñado que el trabajo en equipo requiere tanto competencia técnica como organización y comunicación. Me llevo una visión más completa y realista de lo

que implica desarrollar software en equipo, y eso es algo que valoro especialmente de cara a mi desarrollo profesional.

6.2. Limitaciones y prospectiva

Una vez alcanzados los objetivos establecidos, también nos hemos planteado qué nos gustaría añadir a la aplicación para convertirla en un software realmente robusto y profesional.

Sin duda, además de crear ficheros más limpios y estudiar cómo optimizar su rendimiento, uno de los aspectos que llevaríamos a cabo es la inclusión de una pasarela de pago real. También consideramos que sería útil y más completo si el dashboard contase con su propia bandeja de entrada de correo, de modo que los mensajes de contacto recibidos en la web y los correos electrónicos estuvieran centralizados en un mismo lugar.

En la misma línea, ampliaríamos el sistema de notificaciones para incluir recordatorios de cita por SMS, un canal que, en el sector servicios, presenta tasas de apertura significativamente más altas que el correo electrónico. Igualmente, incorporaríamos un historial de salud por cliente donde registrar alergias, condiciones de piel o contraindicaciones relevantes, una funcionalidad especialmente pertinente en el contexto de un estudio de tatuajes. También sería interesante que cada artista dispusiera de un portfolio integrado dentro de la propia aplicación, vinculado a las citas completadas, lo que aportaría coherencia entre la gestión interna y la vertiente más comercial del negocio.

Otros elementos que consideramos relevantes serían aquellos excluidos por falta de tiempo, como la gestión de stock, que permitiría llevar un seguimiento del material utilizado en cada cita y enviar notificaciones cuando fuera necesario realizar un pedido. También podría ser interesante incluir un apartado para el libro de contabilidad que se exigirá a los trabajadores autónomos en los próximos años, aunque es una funcionalidad compleja que abordaríamos una vez que contemos con mayor experiencia de desarrollo.

7. REFERENCIAS BIBLIOGRÁFICAS

Documentación oficial

- **Amazon Web Services (AWS).** (s.f.). *¿Qué es el ciclo de vida de desarrollo de software (SDLC)?* Recuperado de <https://aws.amazon.com/es/what-is/sdlc/>
- **Bootstrap.** (s.f.). *Bootstrap Documentation.* Recuperado de <https://getbootstrap.com/>
- **IBM.** (s.f.). *¿Qué es el ciclo de vida del desarrollo de software?* Recuperado de <https://www.ibm.com/es-es/think/topics/sdlc>
- **Universidad Internacional de La Rioja (UNIR).** (s.f.). *Temario oficial de la asignatura Desarrollo de Interfaces Web (Temas 1–5).* UNIR.

Herramientas de diseño y desarrollo

- **Adobe.** (s.f.). *Adobe Color (Rueda de colores).* Recuperado de <https://color.adobe.com/es/create/color-wheel>
- **Coolors.** (s.f.). *Generador de paletas de color accesibles.* Recuperado de <https://coolors.co/>
- **Figma.** (s.f.). *Herramienta para diseño de interfaces y wireframes.* Recuperado de <https://figma.com/>
- **Logo.com.** (s.f.). *Generador de logotipos online.* Recuperado de <https://logo.com/>
- **Mailtrap.** (s.f.). *Plataforma de pruebas de envío de correos electrónicos.* Recuperado de <https://mailtrap.io/>
- **NekoCalc.** (s.f.). *Conversor de unidades CSS (PX a REM).* Recuperado de <https://nekocalc.com/>
- **Wappalyzer.** (s.f.). *Technology profiler – Analizador de tecnologías web.* <https://www.wappalyzer.com/>

Recursos visuales y de contenido

- **Dribbble.** (s.f.). *Ejemplos de diseño de formularios de registro.* Recuperado de <https://dribbble.com/search/register-form>
- **Pixabay.** (s.f.). *Banco de imágenes libres de derechos.* Recuperado de <https://pixabay.com/>
- **Skin Design Tattoos.** (s.f.). *Guía de referencia de tamaños de tatuajes.* Recuperado de <https://skindesigntattoos.com/tattoo-sizes/>

Fuentes institucionales y de referencia

- **Agencia Tributaria.** (s.f.). *Portal de la Agencia Tributaria Española – Información sobre facturas simplificadas.* Recuperado de <https://sede.agenciatributaria.gob.es/>
- **Universitat Politècnica de Catalunya (UPC).** (s.f.). *Repositorio UPCommons – Memorias de proyectos de desarrollo de software.* Recuperado de <https://upcommons.upc.edu/>

8. ENLACE AL REPOSITORIO DEL PROYECTO

Para facilitar el acceso al trabajo desarrollado, se incluye un enlace al repositorio empleado para alojarlo:

<https://github.com/pvicSL/Proyecto-Fin-de-Ciclo>

9. ANEXOS

- Anexo A. Guía de estilo (Doc. adjunto).
- Anexo B. Wireflow del sitio web (Doc. adjunto).
- Anexo C. Wireflow del dashboard (Doc. adjunto).
- Anexo D. Mockup sitio web (desktop) (Doc. adjunto).
- Anexo E. Mockup sitio web (móvil) (Doc. adjunto).
- Anexo F. Mockup dashboard - admin (escritorio) (Doc. adjunto).
- Anexo G. Mockup dashboard - admin (móvil) (Doc. adjunto).
- Anexo H. Mockup dashboard – trab. (escritorio) (Doc. adjunto).
- Anexo I. Mockup dashboard – trab. (móvil) (Doc. adjunto).